PmaControl Controllers Documentation

Generated on 2026-03-12

Index
Overview
Documentation Controllers
    Index
About
About
Acl
Acl
Administration
Administration
Agent
Agent
Ai
Ai
Alert
Alert
Alias
Alias
Alter
Alter
Api
Api
Architecture
Architecture
Archives
Archives
Aspirateur
Aspirateur
Audit
Audit
BI
BI
Backup
Backup
Benchmark
Benchmark
Binlog
Binlog
Chartjs
Chartjs
Check
Check
CheckConfig
CheckConfig
CheckDataOnCluster
CheckDataOnCluster
Chemin
Chemin
Cleaner

Cleaner
CleanerTest
CleanerTest
Client
Client
Cluster
Cluster
Color
Color
Common
Common
Compare
Compare
CompareConfig
CompareConfig
Control
Control
Covage
Covage
Crontab
Crontab
Daemon
Daemon
Dashboard
Dashboard
Database
Database
Datamodel
Datamodel
Demo
Demo
DependencyTreeGenerator
DependencyTreeGenerator
Deploy
Deploy
DeployRsaKey
DeployRsaKey
Detail
Detail
Digest
Digest
Disk
Disk
Dns
Dns
Docker
Docker
Dot3
Dot3
Enum
Enum
Environment
Environment
ErrorWeb
ErrorWeb
Event
Event
Explain
Explain

Export
Export
ForeignKey
ForeignKey
Format
Format
Gagman
Gagman
Galera
Galera
GaleraCluster
GaleraCluster
Graph
Graph
GraphicCharter
GraphicCharter
Group
Group
Haproxy
Haproxy
Home
Home
Index
Index
Install
Install
Integrate
Integrate
Job
Job
Layout
Layout
Ldap
Ldap
Listener
Listener
Llm
Llm
Load
Load
Load2
Load2
Log
Log
MasterSlave
MasterSlave
MaxScale
MaxScale
Menu
Menu
Monitoring
Monitoring
Mysql
Mysql
MysqlDatabase
MysqlDatabase
MysqlTable
MysqlTable
MysqlUser

MysqlUser
Mysqlsys
Mysqlsys
Myxplain
Myxplain
Ollama
Ollama
Partition
Partition
Percona
Percona
PhpLiveRegex
PhpLiveRegex
Pid
Pid
Plugin
Plugin
Pmacontrol
Pmacontrol
Pmm
Pmm
PostMortem
PostMortem
ProxySQL
ProxySQL
Query
Query
QueryCache
QueryCache
QueryGraphExtractor
QueryGraphExtractor
Recover
Recover
Release
Release
Replication
Replication
Scan
Scan
Schema
Schema
Server
Server
Site
Site
Slave
Slave
Spider
Spider
Ssh
Ssh
StatementAnalysis
StatementAnalysis
StorageArea
StorageArea
Table
Table
Tag
Tag

Telegram
Telegram
Translation
Translation
Tree
Tree
Tunnel
Tunnel
Upgrade
Upgrade
User
User
Variable
Variable
Ventilateur
Ventilateur
Version
Version
Webservice
Webservice
Worker
Worker

Index

- About
- Acl
- Administration
- Agent
- Ai
- Alert
- Alias
- Alter
- Api
- Architecture
- Archives
- Aspirateur
- Audit
- BI
- Backup
- Benchmark
- Binlog
- Chartjs
- Check
- CheckConfig
- CheckDataOnCluster
- Chemin
- Cleaner
- CleanerTest
- Client
- Cluster
- Color
- Common
- Compare
- CompareConfig
- Control
- Covage
- Crontab

- Daemon
- Dashboard
- Database
- Datamodel
- Demo
- DependencyTreeGenerator
- Deploy
- DeployRsaKey
- Detail
- Digest
- Disk
- Dns
- Docker
- Dot3
- Enum
- Environment
- ErrorWeb
- Event
- Explain
- Export
- ForeignKey
- Format
- Gagman
- Galera
- GaleraCluster
- Graph
- GraphicCharter
- Group
- Haproxy
- Home
- Index
- Install
- Integrate
- Job
- Layout
- Ldap
- Listener
- Llm
- Load
- Load2
- Log
- MasterSlave
- MaxScale
- Menu
- Monitoring
- Mysql
- MysqlDatabase
- MysqlTable
- MysqlUser
- Mysqlsys
- Myxplain
- Ollama
- Partition
- Percona
- PhpLiveRegex
- Pid
- Plugin
- Pmacontrol
- Pmm

- PostMortem
- ProxySQL
- Query
- QueryCache
- QueryGraphExtractor
- Recover
- Release
- Replication
- Scan
- Schema
- Server
- Site
- Slave
- Spider
- Ssh
- StatementAnalysis
- StorageArea
- Table
- Tag
- Telegram
- Translation
- Tree
- Tunnel
- Upgrade
- User
- Variable
- Ventilateur
- Version
- Webservice
- Worker

Overview

Documentation Controllers

- Source: `/srv/www/pmacontrol/App/Controller`
- Documents: 122

Index

- About
- Acl
- Administration
- Agent
- Ai
- Alert
- Alias
- Alter
- Api
- Architecture
- Archives
- Aspirateur
- Audit
- BI
- Backup

- Benchmark
- Binlog
- Chartjs
- Check
- CheckConfig
- CheckDataOnCluster
- Chemin
- Cleaner
- CleanerTest
- Client
- Cluster
- Color
- Common
- Compare
- CompareConfig
- Control
- Covage
- Crontab
- Daemon
- Dashboard
- Database
- Datamodel
- Demo
- DependencyTreeGenerator
- Deploy
- DeployRsaKey
- Detail
- Digest
- Disk
- Dns
- Docker
- Dot3
- Enum
- Environment
- ErrorWeb
- Event
- Explain
- Export
- ForeignKey
- Format
- Gagman
- Galera
- GaleraCluster
- Graph
- GraphicCharter
- Group
- Haproxy
- Home
- Index
- Install
- Integrate
- Job
- Layout
- Ldap
- Listener
- Llm
- Load
- Load2
- Log

- MasterSlave
- MaxScale
- Menu
- Monitoring
- Mysql
- MysqlDatabase
- MysqlTable
- MysqlUser
- Mysqlsys
- Myxplain
- Ollama
- Partition
- Percona
- PhpLiveRegex
- Pid
- Plugin
- Pmacontrol
- Pmm
- PostMortem
- ProxySQL
- Query
- QueryCache
- QueryGraphExtractor
- Recover
- Release
- Replication
- Scan
- Schema
- Server
- Site
- Slave
- Spider
- Ssh
- StatementAnalysis
- StorageArea
- Table
- Tag
- Telegram
- Translation
- Tree
- Tunnel
- Upgrade
- User
- Variable
- Ventilateur
- Version
- Webservice
- Worker

About

About

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/About.php`
- `index()`: Render about state through `index`.
- `getResult($sql)`: Retrieve about state through `getResult`.

Acl

Acl

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Acl.php`
- `index()`: Render acl state through `index`.
- `check()`: Handle acl state through `check`.

Administration

Administration

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Administration.php`
- `test()`: Handle administration state through `test`.

Agent

Agent

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Agent.php`
- `before($param)`: Prepare agent state through `before`.
- `start($param)`: Handle agent state through `start`.
- `stop($param)`: Handle agent state through `stop`.
- `launch($params)`: Handle agent state through `launch`.
- `updateServerList()`: Update agent state through `updateServerList`.
- `logs($param)`: Handle agent state through `logs`.
- `tailCustom($filepath, $lines, $adaptive)`: Handle agent state through `tailCustom`.
- `check_daemon()`: Handle agent state through `check_daemon`.
- `check_queue($param)`: Handle agent state through `check_queue`.

Ai

Ai

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Ai.php`
- `index($param)`: Render ai state through `index`.

Alert

Alert

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Alert.php`
- `check($date, $id_servers)`: Handle alert state through `check`.
- `reboot($param)`: Handle alert state through `reboot`.
- `test()`: Handle alert state through `test`.

Alias

Alias

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Alias.php`
- `index()`: Render alias state through `index`.
- `updateAlias($param)`: Update alias state through `updateAlias`.
- `delete($param)`: Delete alias state through `delete`.
- `getExtraction($param)`: Retrieve alias state through `getExtraction`.
- `getIdfromDns($param)`: Retrieve alias state through `getIdfromDns`.
- `getIdfromHostname($param)`: Retrieve alias state through `getIdfromHostname`.
- `addHostname($param)`: Create alias state through `addHostname`.
- `addAliasFromHostname($param)`: Create alias state through `addAliasFromHostname`.
- `upsertAliasDns($param)`: Handle alias state through `upsertAliasDns`.
- `addAliasFromSshIps($param)`: Create alias state through `addAliasFromSshIps`.
- `extractIpList($raw)`: Handle alias state through `extractIpList`.
- `addAliasFromWsrepNodeAddress($param)`: Create alias state through `addAliasFromWsrepNodeAddress`.
- `clearAliasDnsCache()`: Handle alias state through `clearAliasDnsCache`.

Alter

Alter

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Alter.php`
- `dropsp($param)`: Handle alter state through `dropsp`.
- `slave($param)`: Handle alter state through `slave`.
- `user($param)`: Handle alter state through `user`.
- `dropRoot($param)`: Handle alter state through `dropRoot`.

Api

Api

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Api.php`
- `config($param)`: Handle api state through `config`.
- `openApi($param)`: Handle api state through `openApi`.
- `getResourceMap()`: Retrieve api state through `getResourceMap`.
- `getResourceDefinition($resource)`: Retrieve api state through `getResourceDefinition`.
- `normalizePayload($resource, $payload, $isUpdate)`: Handle api state through `normalizePayload`.
- `getOpenApiDocument()`: Retrieve api state through `getOpenApiDocument`.
- `normalizeBoolean($value)`: Handle api state through `normalizeBoolean`.
- `handleGet($db, $definition, $id)`: Handle api state through `handleGet`.
- `handleCreate($db, $resource, $definition, $payload)`: Handle api state through `handleCreate`.
- `handleUpdate($db, $resource, $definition, $id, $payload)`: Handle api state through `handleUpdate`.
- `handleDelete($db, $definition, $id)`: Handle api state through `handleDelete`.
- `readJsonInput()`: Handle api state through `readJsonInput`.
- `respondJson($payload, $status)`: Handle api state through `respondJson`.

Architecture

Architecture

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Architecture.php`
- `index($param)`: Render architecture state through `index`.
- `view($param)`: Handle architecture state through `view`.

Archives

Archives

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Archives.php`
- `index($param)`: Render archives state through `index`.
- `file_available($param)`: Handle archives state through `file_available`.
- `load($param)`: Handle archives state through `load`.
- `history($param)`: Handle archives state through `history`.
- `restore($param)`: Handle archives state through `restore`.
- `menu($param)`: Handle archives state through `menu`.
- `testPid()`: Handle archives state through `testPid`.
- `before($param)`: Prepare archives state through `before`.
- `log($level, $type, $msg)`: Handle archives state through `log`.
- `getUser()`: Retrieve archives state through `getUser`.
- `detail($param)`: Handle archives state through `detail`.
- `load_archive($param)`: Handle archives state through `load_archive`.
- `format($lines, $id_cleaner)`: Handle archives state through `format`.
- `setColor($type)`: Handle archives state through `setColor`.
- `hexToRgb($colorName)`: Handle archives state through `hexToRgb`.

Aspirateur

Aspirateur

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Aspirateur.php`
- `before($param)`: Prepare aspirateur state through `before`.
- `tryMysqlConnection($param)`: Handle aspirateur state through `tryMysqlConnection`.
- `upsertVipServerRoute($id_mysql_server, $dns, $ip, $newActual)`: Handle aspirateur state through `upsertVipServerRoute`.
- `resolveVipIp($connectionHost)`: Handle aspirateur state through `resolveVipIp`.
- `resolveVipDestinationId($id_mysql_server, $vipCandidates, $port)`: Handle aspirateur state through `resolveVipDestinationId`.
- `resolveVipDestinationIdFromSshMetric($id_mysql_server, $normalizedTarget, $metricSelector, $fieldName)`: Handle aspirateur state through `resolveVipDestinationIdFromSshMetric`.
- `resolveVipDestinationIdFromAliasDns($id_mysql_server, $normalizedCandidates, $port)`: Handle aspirateur state through `resolveVipDestinationIdFromAliasDns`.
- `extractVipCandidatesFromRawValue($raw)`: Handle aspirateur state through `extractVipCandidatesFromRawValue`.
- `flattenVipRawValue($value, $flat)`: Handle aspirateur state through `flattenVipRawValue`.
- `normalizeVipCandidate($value)`: Handle aspirateur state through `normalizeVipCandidate`.
- `allocate_shared_storage($name, $separator)`: Handle aspirateur state through `allocate_shared_storage`.
- `trySshConnection($param)`: Handle aspirateur state through `trySshConnection`.
- `getHardware($ssh)`: Retrieve aspirateur state through `getHardware`.
- `getStats($ssh, $mysqlContext)`: Retrieve aspirateur state through `getStats`.
- `getInstantCpuUsage($ssh, $intervalSeconds)`: Retrieve aspirateur state through `getInstantCpuUsage`.
- `parseProcStatSnapshot($raw)`: Handle aspirateur state through `parseProcStatSnapshot`.
- `computeCpuUsageDelta($start, $end)`: Handle aspirateur state through `computeCpuUsageDelta`.

- `getMysqlDatadirContext($id_mysql_server)`: Retrieve aspirateur state through `getMysqlDatadirContext`.
- `getMysqlDatadirStats($ssh, $mysqlContext)`: Retrieve aspirateur state through `getMysqlDatadirStats`.
- `getMysqlDatadirCleanSize($ssh, $datadir, $logBinBasename, $relayLogBasename)`: Retrieve aspirateur state through `getMysqlDatadirCleanSize`.
- `getSstElapsedSeconds($ssh)`: Retrieve aspirateur state through `getSstElapsedSeconds`.
- `binaryLog($param)`: Handle aspirateur state through `binaryLog`.
- `getArbitrator()`: Retrieve aspirateur state through `getArbitrator`.
- `getDatabase($mysql_tested)`: Retrieve aspirateur state through `getDatabase`.
- `getSwap($membrut)`: Retrieve aspirateur state through `getSwap`.
- `getWsrep($param)`: Retrieve aspirateur state through `getWsrep`.
- `getLockingQueries($param)`: Retrieve aspirateur state through `getLockingQueries`.
- `getProxySQL($param)`: Retrieve aspirateur state through `getProxySQL`.
- `testProxy($param)`: Handle aspirateur state through `testProxy`.
- `setService($id_mysql_server, $ping, $error_msg, $available, $type)`: Handle aspirateur state through `setService`.
- `debug($param)`: Handle aspirateur state through `debug`.
- `getSchema($id_mysql_server)`: Retrieve aspirateur state through `getSchema`.
- `keepConfigFile($param)`: Handle aspirateur state through `keepConfigFile`.
- `testproxysql($param)`: Handle aspirateur state through `testproxysql`.
- `exportData($id_mysql_server, $ts_file, $data, $check_data, $separator)`: Handle aspirateur state through `exportData`.
- `isDataModified($id_mysql_server, $ts_file, $data)`: Handle aspirateur state through `isDataModified`.
- `test2($param)`: Handle aspirateur state through `test2`.
- `after($param)`: Handle aspirateur state through `after`.
- `isValidStruc($array)`: Handle aspirateur state through `isValidStruc`.
- `getMysqlLatencyByQuery($name_server)`: Retrieve aspirateur state through `getMysqlLatencyByQuery`.
- `getInnodbMetrics($name_server)`: Retrieve aspirateur state through `getInnodbMetrics`.
- `tryProxySqlConnection($param)`: Handle aspirateur state through `tryProxySqlConnection`.

- `getElemFromTable($param)`: Retrieve aspirateur state through `getElemFromTable`.
- `getTableElems($id_mysql_server, $database, $table)`: Retrieve aspirateur state through `getTableElems`.
- `getTableExist($id_mysql_server, $database, $table)`: Retrieve aspirateur state through `getTableExist`.
- `getTableFromProxySQL($id_proxysql)`: Retrieve aspirateur state through `getTableFromProxySQL`.
- `getPsMemory($id_mysql_server)`: Retrieve aspirateur state through `getPsMemory`.
- `getDigest($param)`: Retrieve aspirateur state through `getDigest`.
- `getProcesslist($db_link, $isSingleStore)`: Retrieve aspirateur state through `getProcesslist`.
- `array_values_to_lowercase($array)`: Handle aspirateur state through `array_values_to_lowercase`.
- `getTables($param)`: Retrieve aspirateur state through `getTables`.
- `getCreateTables($param)`: Retrieve aspirateur state through `getCreateTables`.
- `eachHour($param)`: Handle aspirateur state through `eachHour`.
- `getDisks($param)`: Retrieve aspirateur state through `getDisks`.
- `getVelocity($name_server)`: Retrieve aspirateur state through `getVelocity`.
- `detectDouble($param)`: Handle aspirateur state through `detectDouble`.
- `find_duplicate_server_uids($servers)`: Handle aspirateur state through `find_duplicate_server_uids`.
- `tryMaxScaleConnection($param)`: Handle aspirateur state through `tryMaxScaleConnection`.

Audit

Audit

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Audit.php`
- `getuser($param)`: Retrieve audit state through `getuser`.
- `general_log($param)`: Handle audit state through `general_log`.
- `scp($param)`: Handle audit state through `scp`.
- `export($param)`: Handle audit state through `export`.
- `server($param)`: Handle audit state through `server`.
- `upload($param)`: Handle audit state through `upload`.
- `callXmlRpc($xmlContent, $xmlrpcUrl, $cookieJar)`: Handle audit state through `callXmlRpc`.
- `recommandation($param)`: Handle audit state through `recommandation`.
- `queryCache($param)`: Handle audit state through `queryCache`.
- `all($param)`: Handle audit state through `all`.
- `cluster($param)`: Handle audit state through `cluster`.
- `byCluster($param)`: Handle audit state through `byCluster`.
- `get_common_parts($servers)`: Retrieve audit state through `get_common_parts`.
- `retirerChiffreEtSeparateurFin($chaine)`: Handle audit state through `retirerChiffreEtSeparateurFin`.
- `getQueryOnCluster($param)`: Retrieve audit state through `getQueryOnCluster`.
- `getTableWithoutFk($param)`: Retrieve audit state through `getTableWithoutFk`.
- `displayTable($param)`: Handle audit state through `displayTable`.
- `aggregate($param)`: Handle audit state through `aggregate`.
- `getAutoInc($param)`: Retrieve audit state through `getAutoInc`.
- `getIndex($param)`: Retrieve audit state through `getIndex`.
- `getRedundantIndex($param)`: Retrieve audit state through `getRedundantIndex`.
- `getRedundantAlter($param)`: Retrieve audit state through `getRedundantAlter`.
- `getUnusedIndex($param)`: Retrieve audit state through `getUnusedIndex`.

- `getConfig($param)`: Retrieve audit state through `getConfig`.

BI

BI

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/BI.php`
- `searchField($param)`: Handle b i state through `searchField`.
- `createServer($param)`: Create b i state through `createServer`.
- `rapport($param)`: Handle b i state through `rapport`.
- `createTableSpider($param)`: Create b i state through `createTableSpider`.
- `getBackend($param)`: Retrieve b i state through `getBackend`.
- `getCreateTable($param)`: Retrieve b i state through `getCreateTable`.
- `changeToSpider($param)`: Handle b i state through `changeToSpider`.

Backup

Backup

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Backup.php`
- `before($param)`: Prepare backup state through `before`.
- `sendKeyPub()`: Handle backup state through `sendKeyPub`.
- `checkDirectory($dir)`: Handle backup state through `checkDirectory`.
- `cmd($cmd)`: Handle backup state through `cmd`.
- `compress_dump()`: Handle backup state through `compress_dump`.
- `backupUser()`: Handle backup state through `backupUser`.
- `listing()`: Retrieve backup state through `listing`.
- `getDump($param)`: Retrieve backup state through `getDump`.
- `settings()`: Handle backup state through `settings`.
- `getDatabaseByServer($param)`: Retrieve backup state through `getDatabaseByServer`.
- `getServerByName($param)`: Retrieve backup state through `getServerByName`.
- `getServerByIp($server)`: Retrieve backup state through `getServerByIp`.
- `saveDb($param)`: Update backup state through `saveDb`.
- `deleteShedule($param)`: Delete backup state through `deleteShedule`.
- `toggleShedule($param)`: Toggle backup state through `toggleShedule`.
- `mysqldump($backup)`: Handle backup state through `mysqldump`.

Benchmark

Benchmark

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Benchmark.php`
- `before($param)`: Prepare benchmark state through `before`.
- `run($param)`: Handle benchmark state through `run`.
- `getQueriesPerformedRead($input_lines)`: Retrieve benchmark state through `getQueriesPerformedRead`.
- `getQueriesPerformedWrite($input_lines)`: Retrieve benchmark state through `getQueriesPerformedWrite`.
- `getQueriesPerformedOther($input_lines)`: Retrieve benchmark state through `getQueriesPerformedOther`.
- `getQueriesPerformedTotal($input_lines)`: Retrieve benchmark state through `getQueriesPerformedTotal`.
- `getTransactions($input_lines)`: Retrieve benchmark state through `getTransactions`.
- `getErrors($input_lines)`: Retrieve benchmark state through `getErrors`.
- `getTotalTime($input_lines)`: Retrieve benchmark state through `getTotalTime`.
- `getReponseTimeMin($input_lines)`: Retrieve benchmark state through `getReponseTimeMin`.
- `getReponseTimeMax($input_lines)`: Retrieve benchmark state through `getReponseTimeMax`.
- `getReponseTimeAvg($input_lines)`: Retrieve benchmark state through `getReponseTimeAvg`.
- `getReponseTime95percent($input_lines)`: Retrieve benchmark state through `getReponseTime95percent`.
- `testMoc($param)`: Handle benchmark state through `testMoc`.
- `moc()`: Handle benchmark state through `moc`.
- `moc2()`: Handle benchmark state through `moc2`.
- `moc3()`: Handle benchmark state through `moc3`.
- `index($param)`: Render benchmark state through `index`.
- `testError($input_lines)`: Handle benchmark state through `testError`.
- `install()`: Handle benchmark state through `install`.
- `uninstall()`: Handle benchmark state through `uninstall`.

- `graph()`: Handle benchmark state through `graph`.
- `config()`: Handle benchmark state through `config`.
- `bench($param)`: Handle benchmark state through `bench`.
- `current()`: Handle benchmark state through `current`.
- `getFilter()`: Retrieve benchmark state through `getFilter`.
- `queue($param)`: Handle benchmark state through `queue`.
- `debug($string)`: Handle benchmark state through `debug`.
- `getSysbenchVersion($param)`: Retrieve benchmark state through `getSysbenchVersion`.
- `getScriptLua($param)`: Retrieve benchmark state through `getScriptLua`.
- `getDirectoryLua($param)`: Retrieve benchmark state through `getDirectoryLua`.
- `getLua()`: Retrieve benchmark state through `getLua`.

Binlog

Binlog

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Binlog.php`
- `index()`: Render binlog state through `index`.
- `before($param)`: Prepare binlog state through `before`.
- `add()`: Create binlog state through `add`.
- `max()`: Handle binlog state through `max`.
- `getMaxBinlogSize($param)`: Retrieve binlog state through `getMaxBinlogSize`.
- `view($param)`: Handle binlog state through `view`.
- `search($param)`: Handle binlog state through `search`.
- `backupAll($param)`: Handle binlog state through `backupAll`.
- `backup($param)`: Handle binlog state through `backup`.
- `backupServer($param)`: Handle binlog state through `backupServer`.
- `purgeAll($param)`: Handle binlog state through `purgeAll`.
- `purgeServer($param)`: Handle binlog state through `purgeServer`.
- `liste($param)`: Retrieve binlog state through `liste`.
- `getBinlog($param)`: Retrieve binlog state through `getBinlog`.
- `binlog2sql($param)`: Handle binlog state through `binlog2sql`.
- `getLastSqlError($param)`: Retrieve binlog state through `getLastSqlError`.

Chartjs

Chartjs

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Chartjs.php`
- `lineBasic($param)`: Handle chartjs state through `lineBasic`.

Check

Check

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Check.php`
- `uptimeDecrease($servers)`: Handle check state through `uptimeDecrease`.

CheckConfig

CheckConfig

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/CheckConfig.php`
- `index($param)`: Render check config state through `index`.
- `getDatabasesByServers($param)`: Retrieve check config state through `getDatabasesByServers`.
- `getDbLinkFromId($id_db)`: Retrieve check config state through `getDbLinkFromId`.
- `see($param)`: Handle check config state through `see`.

CheckDataOnCluster

CheckDataOnCluster

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/CheckDataOnCluster.php`
- `index($param)`: Render check data on cluster state through `index`.
- `getDatabasesByServers($param)`: Retrieve check data on cluster state through `getDatabasesByServers`.
- `getDbLinkFromId($id_db)`: Retrieve check data on cluster state through `getDbLinkFromId`.
- `array_diff_assoc_recursive($array1, $array2)`: Handle check data on cluster state through `array_diff_assoc_recursive`.
- `liste_combinaison($list)`: Retrieve check data on cluster state through `liste_combinaison`.
- `perm($nbrs)`: Handle check data on cluster state through `perm`.
- `see($param)`: Handle check data on cluster state through `see`.

Chemin

Chemin

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Chemin.php`
- `possibilite($param)`: Handle chemin state through `possibilite`.
- `getNameMysqlServer($id)`: Retrieve chemin state through `getNameMysqlServer`.
- `getPaths($table, $table2, $database)`: Retrieve chemin state through `getPaths`.

Cleaner

Cleaner

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Cleaner.php`
- `statistics($param)`: Handle cleaner state through `statistics`.
- `getMsgStartDaemon($ob)`: Retrieve cleaner state through `getMsgStartDaemon`.
- `showDaemon()`: Handle cleaner state through `showDaemon`.
- `index($param)`: Render cleaner state through `index`.
- `treatment($param)`: Handle cleaner state through `treatment`.
- `detail($param)`: Handle cleaner state through `detail`.
- `add($param)`: Create cleaner state through `add`.
- `getDatabaseByServer($param)`: Retrieve cleaner state through `getDatabaseByServer`.
- `getTableByDatabase($param)`: Retrieve cleaner state through `getTableByDatabase`.
- `getColumnByTable($param)`: Retrieve cleaner state through `getColumnByTable`.
- `delete($param)`: Delete cleaner state through `delete`.
- `settings($param)`: Handle cleaner state through `settings`.
- `launch($param)`: Handle cleaner state through `launch`.
- `start($param)`: Handle cleaner state through `start`.
- `stop($param)`: Handle cleaner state through `stop`.
- `restart($param)`: Handle cleaner state through `restart`.
- `isRunning($pid)`: Handle cleaner state through `isRunning`.
- `stats_for_log($data)`: Handle cleaner state through `stats_for_log`.
- `getTableImpacted($param)`: Retrieve cleaner state through `getTableImpacted`.
- `checkFileToPush($path)`: Handle cleaner state through `checkFileToPush`.
- `compressAndCrypt($file, $is_cryted)`: Handle cleaner state through `compressAndCrypt`.
- `getFileinfo($filename)`: Retrieve cleaner state through `getFileinfo`.
- `cryptFile($file_name)`: Handle cleaner state through `cryptFile`.
- `decryptFile($file_name)`: Handle cleaner state through `decryptFile`.

- `compressFile($path_file)`: Handle cleaner state through `compressFile`.
- `unCompressFile($path_file)`: Handle cleaner state through `unCompressFile`.
- `uncc($param)`: Handle cleaner state through `uncc`.
- `purge_clean_db($param)`: Handle cleaner state through `purge_clean_db`.
- `sig_handler($signo)`: Handle cleaner state through `sig_handler`.
- `get_id_cleaner($param)`: Retrieve cleaner state through `get_id_cleaner`.
- `purge()`: Handle cleaner state through `purge`.
- `createTemporaryTable($table)`: Create cleaner state through `createTemporaryTable`.
- `feedDeleteTableWithFk()`: Handle cleaner state through `feedDeleteTableWithFk`.
- `createAllTemporaryTable()`: Create cleaner state through `createAllTemporaryTable`.
- `getTableError()`: Retrieve cleaner state through `getTableError`.
- `getRealForeignKeys()`: Retrieve cleaner state through `getRealForeignKeys`.
- `getVirtualForeignKeys()`: Retrieve cleaner state through `getVirtualForeignKeys`.
- `getOrderBy($param)`: Retrieve cleaner state through `getOrderBy`.
- `delete_rows()`: Delete cleaner state through `delete_rows`.
- `setAffectedRows($table)`: Handle cleaner state through `setAffectedRows`.
- `getAffectedTables()`: Retrieve cleaner state through `getAffectedTables`.
- `removeTableNotImpacted($fks)`: Delete cleaner state through `removeTableNotImpacted`.
- `exportToFile($table)`: Handle cleaner state through `exportToFile`.
- `testDirectory($path)`: Handle cleaner state through `testDirectory`.
- `__sleep()`: Handle cleaner state through `__sleep`.
- `get_rows($result)`: Retrieve cleaner state through `get_rows`.
- `printableBitValue($value, $length)`: Handle cleaner state through `printableBitValue`.
- `getImpactedTable()`: Retrieve cleaner state through `getImpactedTable`.
- `before($param)`: Prepare cleaner state through `before`.
- `pushArchive($param)`: Handle cleaner state through `pushArchive`.
- `getIdStorageArea($id_cleaner)`: Retrieve cleaner state through `getIdStorageArea`.

- `init()`: Handle cleaner state through `init`.
- `generateCreateTable()`: Handle cleaner state through `generateCreateTable`.
- `cacheDdlOnDisk()`: Handle cleaner state through `cacheDdlOnDisk`.
- `initFileWithCreateTable($file)`: Handle cleaner state through `initFileWithCreateTable`.
- `cleanTableFromNoNeedConstraint($tableCreate)`: Handle cleaner state through `cleanTableFromNoNeedConstraint`.
- `cacheComStatus()`: Handle cleaner state through `cacheComStatus`.
- `compareComStatus()`: Handle cleaner state through `compareComStatus`.
- `sighup()`: Handle cleaner state through `sighup`.
- `getPrimaryKey($table, $database)`: Retrieve cleaner state through `getPrimaryKey`.
- `end_loop()`: Handle cleaner state through `end_loop`.
- `initDdlOnDisk()`: Handle cleaner state through `initDdlOnDisk`.
- `compareDdl()`: Handle cleaner state through `compareDdl`.
- `compareTables()`: Handle cleaner state through `compareTables`.
- `compareTable($table)`: Handle cleaner state through `compareTable`.
- `edit($param)`: Handle cleaner state through `edit`.
- `install()`: Handle cleaner state through `install`.
- `uninstall()`: Handle cleaner state through `uninstall`.
- `view($param)`: Handle cleaner state through `view`.
- `logs($param)`: Handle cleaner state through `logs`.
- `details($param)`: Handle cleaner state through `details`.
- `menu($param)`: Handle cleaner state through `menu`.
- `format($lines, $id_cleaner)`: Handle cleaner state through `format`.
- `setColor($type)`: Handle cleaner state through `setColor`.
- `label($text)`: Handle cleaner state through `label`.
- `hexToRgb($colorName)`: Handle cleaner state through `hexToRgb`.
- `getUser($id)`: Retrieve cleaner state through `getUser`.
- `log($level, $type, $msg)`: Handle cleaner state through `log`.
- `getrgba($label, $alpha)`: Retrieve cleaner state through `getrgba`.
- `impacted($param)`: Handle cleaner state through `impacted`.
- `setCacheFile()`: Handle cleaner state through `setCacheFile`.
- `addChild($fks, $ref_table, $childs)`: Create cleaner state through `addChild`.
- `filterFkWithChildren($children, $foreign_keys)`: Handle cleaner state through `filterFkWithChildren`.

- `getForeignKeys()`: Retrieve cleaner state through `getForeignKeys`.
- `generateMock($param)`: Handle cleaner state through `generateMock`.
- `skipReplication($db)`: Handle cleaner state through `skipReplication`.
- `testOrder($param)`: Handle cleaner state through `testOrder`.
- `getOrderBy2($param)`: Retrieve cleaner state through `getOrderBy2`.
- `dropFk($child, $parent)`: Handle cleaner state through `dropFk`.
- `detectCircularDefinition($param)`: Handle cleaner state through `detectCircularDefinition`.
- `getCircularMulti($param)`: Retrieve cleaner state through `getCircularMulti`.
- `getPath($table, $table2, $db)`: Retrieve cleaner state through `getPath`.
- `testOneToOne($way, $order)`: Handle cleaner state through `testOneToOne`.
- `testFk($table_a, $table_b)`: Handle cleaner state through `testFk`.
- `comptage($param)`: Handle cleaner state through `comptage`.
- `feedCircularFk($table)`: Handle cleaner state through `feedCircularFk`.
- `setDebug($param)`: Handle cleaner state through `setDebug`.

CleanerTest

CleanerTest

- Type: class
- Namespace: `App\Controller\Test`
- Source: `App/Controller/Test/CleanerTest.php`
- `mydb()`: Handle cleaner test state through `mydb`.
- `testPushAndPop()`: Handle cleaner test state through `testPushAndPop`.
- `testGetOrderBy()`: Handle cleaner test state through `testGetOrderBy`.

Client

Client

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Client.php`
- `index()`: Render client state through `index`.
- `add()`: Create client state through `add`.
- `update($param)`: Update client state through `update`.
- `toggleMonitoring($param)`: Toggle client state through `toggleMonitoring`.
- `delete($param)`: Delete client state through `delete`.

Cluster

Cluster

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Cluster.php`
- `before($param)`: Prepare cluster state through `before`.
- `svg($param)`: Handle cluster state through `svg`.
- `replay($param)`: Handle cluster state through `replay`.

Color

Color

- Type: class
- Namespace: `App\Library`
- Source: `App/Library/Color.php`
- `setFontColor($type)`: Handle color state through `setFontColor`.
- `setBackgroundColor($type)`: Handle color state through `setBackgroundColor`.
- `testColor($param)`: Handle color state through `testColor`.
- `getFontColor($b_color)`: Retrieve color state through `getFontColor`.
- `isDark()`: Handle color state through `isDark`.

Common

Common

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Common.php`
- `index()`: Render common state through `index`.
- `displayClientEnvironment($param)`: Handle common state through `displayClientEnvironment`.
- `remove($array)`: Delete common state through `remove`.
- `getDatabaseByServer($param)`: Retrieve common state through `getDatabaseByServer`.
- `getDbLinkFromId($id_db)`: Retrieve common state through `getDbLinkFromId`.
- `getTableByServerAndDatabase($param)`: Retrieve common state through `getTableByServerAndDatabase`.
- `getSelectServerAvailable($param)`: Retrieve common state through `getSelectServerAvailable`.
- `getTsVariables($param)`: Retrieve common state through `getTsVariables`.
- `getTsVariableJson($param)`: Retrieve common state through `getTsVariableJson`.
- `getTagByServer($param)`: Retrieve common state through `getTagByServer`.
- `getTagArray($db)`: Retrieve common state through `getTagArray`.
- `getAvailable($param)`: Retrieve common state through `getAvailable`.

Compare

Compare

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Compare.php`
- `index($params)`: Render compare state through `index`.
- `checkConfig($id_server1, $db1, $id_server2, $db2)`: Handle compare state through `checkConfig`.
- `analyse($id_server1, $db1, $id_server2, $db2)`: Handle compare state through `analyse`.
- `compareTable($original, $compare, $data)`: Handle compare state through `compareTable`.
- `execMulti($queries, $db_link)`: Handle compare state through `execMulti`.
- `compareListObject($db1, $db2, $type_object)`: Handle compare state through `compareListObject`.
- `menu($param)`: Handle compare state through `menu`.
- `generateGet()`: Handle compare state through `generateGet`.
- `getObjectDiff($param)`: Retrieve compare state through `getObjectDiff`.
- `compareObject($db1, $db2, $data)`: Handle compare state through `compareObject`.
- `getDatabaseByServer($param)`: Retrieve compare state through `getDatabaseByServer`.
- `getDbLinkFromId($id_db)`: Retrieve compare state through `getDbLinkFromId`.

CompareConfig

CompareConfig

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/CompareConfig.php`
- `index($param)`: Render compare config state through `index`.
- `checkConfig($id_server1, $db1, $id_server2, $db2)`: Handle compare config state through `checkConfig`.
- `analyse($id_server1, $db1, $id_server2, $db2)`: Handle compare config state through `analyse`.
- `compareTable($original, $compare, $data)`: Handle compare config state through `compareTable`.
- `execMulti($queries, $db_link)`: Handle compare config state through `execMulti`.
- `compareListObject($db1, $db2, $type_object)`: Handle compare config state through `compareListObject`.
- `menu($param)`: Handle compare config state through `menu`.
- `generateGet()`: Handle compare config state through `generateGet`.
- `getObjectDiff($param)`: Retrieve compare config state through `getObjectDiff`.
- `compareObject($db1, $db2, $data)`: Handle compare config state through `compareObject`.
- `getDatabaseByServer($param)`: Retrieve compare config state through `getDatabaseByServer`.
- `getDbLinkFromId($id_db)`: Retrieve compare config state through `getDbLinkFromId`.

Control

Control

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Control.php`
- `checkSize($param)`: Handle control state through `checkSize`.
- `before($param)`: Prepare control state through `before`.
- `selectEngine()`: Handle control state through `selectEngine`.
- `addPartition($param)`: Create control state through `addPartition`.
- `makeCombinaison()`: Handle control state through `makeCombinaison`.
- `dropPartition($param)`: Handle control state through `dropPartition`.
- `getMinMaxPartition()`: Retrieve control state through `getMinMaxPartition`.
- `getToDays($param)`: Retrieve control state through `getToDays`.
- `service($param)`: Handle control state through `service`.
- `dropTsTable($param)`: Handle control state through `dropTsTable`.
- `createTsTable()`: Create control state through `createTsTable`.
- `rebuildAll($param)`: Handle control state through `rebuildAll`.
- `statistique($param)`: Handle control state through `statistique`.
- `getDates()`: Retrieve control state through `getDates`.
- `dropFile($diretory)`: Handle control state through `dropFile`.
- `dropAllFile($param)`: Handle control state through `dropAllFile`.
- `refreshVariable($param)`: Handle control state through `refreshVariable`.
- `purgefrm($param)`: Handle control state through `purgefrm`.
- `truncateTsVariable()`: Handle control state through `truncateTsVariable`.
- `truncateTsMaxDate()`: Handle control state through `truncateTsMaxDate`.
- `truncateTsFile()`: Handle control state through `truncateTsFile`.
- `delMd5File($param)`: Handle control state through `delMd5File`.
- `rocksdbCompact($param)`: Handle control state through `rocksdbCompact`.
- `generateAllTables($param)`: Handle control state through `generateAllTables`.

- `purgeAll($param)`: Handle control state through `purgeAll`.

Covage

Covage

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Covage.php`
- `creationTableSpider()`: Handle covage state through `creationTableSpider`.
- `getEnvs()`: Retrieve covage state through `getEnvs`.
- `all($param)`: Handle covage state through `all`.
- `drop()`: Handle covage state through `drop`.
- `diff($param)`: Handle covage state through `diff`.
- `getPrimaryKey($database, $table)`: Retrieve covage state through `getPrimaryKey`.
- `reprise($param)`: Handle covage state through `reprise`.
- `saveRef()`: Update covage state through `saveRef`.
- `getFields($database, $table)`: Retrieve covage state through `getFields`.
- `createTableSpider($param)`: Create covage state through `createTableSpider`.
- `createDbLink($param)`: Create covage state through `createDbLink`.
- `getInfoFromBackend($backend, $id_mysql_server)`: Retrieve covage state through `getInfoFromBackend`.
- `toRename($param)`: Handle covage state through `toRename`.
- `toRenameRollback($param)`: Handle covage state through `toRenameRollback`.
- `convertToUtf8($param)`: Handle covage state through `convertToUtf8`.

Crontab

Crontab

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Crontab.php`
- `index()`: Render crontab state through `index`.
- `admin_crontab()`: Handle crontab state through `admin_crontab`.
- `view()`: Handle crontab state through `view`.
- `monitor($param)`: Handle crontab state through `monitor`.

Daemon

Daemon

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Daemon.php`
- `index($param)`: Render daemon state through `index`.
- `startAll($param)`: Handle daemon state through `startAll`.
- `stopAll($param)`: Handle daemon state through `stopAll`.
- `manageDaemon($commande)`: Handle daemon state through `manageDaemon`.
- `update()`: Update daemon state through `update`.
- `refresh($param)`: Handle daemon state through `refresh`.
- `getStatitics($param)`: Retrieve daemon state through `getStatitics`.
- `getAllProcessPhp($param)`: Retrieve daemon state through `getAllProcessPhp`.

Dashboard

Dashboard

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Dashboard.php`
- `json($param)`: Handle dashboard state through `json`.
- `hitRatio($param)`: Handle dashboard state through `hitRatio`.
- `ratioTable($param)`: Handle dashboard state through `ratioTable`.
- `ratioLockTable($param)`: Handle dashboard state through `ratioLockTable`.
- `ratioThreadCache($param)`: Handle dashboard state through `ratioThreadCache`.
- `ratioOpenFile($param)`: Handle dashboard state through `ratioOpenFile`.

Database

Database

- Type: class
- Namespace: `App\Library`
- Source: `App/Library/Database.php`
- `getTagSize($size)`: Retrieve database state through `getTagSize`.
- `emptyDatabase($param)`: Handle database state through `emptyDatabase`.

Datamodel

Datamodel

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Datamodel.php`
- `index()`: Render datamodel state through `index`.
- `add()`: Create datamodel state through `add`.

Demo

Demo

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Demo.php`
- `index()`: Render demo state through `index`.
- `getTestServer($param)`: Retrieve demo state through `getTestServer`.
- `AssociateServerByLevel($param)`: Handle demo state through `AssociateServerByLevel`.
- `compareVersions($a, $b)`: Handle demo state through `compareVersions`.
- `compareDigit($a, $b)`: Handle demo state through `compareDigit`.
- `obtenirPlusGrandChiffre($s)`: Handle demo state through `obtenirPlusGrandChiffre`.
- `generatePair($param)`: Handle demo state through `generatePair`.
- `configMasterSlave($param)`: Handle demo state through `configMasterSlave`.
- `randomPassword()`: Handle demo state through `randomPassword`.
- `install($param)`: Handle demo state through `install`.
- `createSakila($param)`: Create demo state through `createSakila`.
- `createInstanceMariaDB($param)`: Create demo state through `createInstanceMariaDB`.
- `dropDemo($param)`: Handle demo state through `dropDemo`.

DependencyTreeGenerator

DependencyTreeGenerator

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/DependencyTreeGenerator.php`
- `init($directory)`: Handle dependency tree generator state through `init`.
- `loadFiles($directory)`: Handle dependency tree generator state through `loadFiles`.
- `parseMethodCalls($className, $methodName)`: Handle dependency tree generator state through `parseMethodCalls`.

Deploy

Deploy

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Deploy.php`
- `index()`: Render deploy state through `index`.
- `execute($param)`: Handle deploy state through `execute`.

DeployRsaKey

DeployRsaKey

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/DeployRsaKey.php`
- `index()`: Render deploy rsa key state through `index`.
- `getFilter()`: Retrieve deploy rsa key state through `getFilter`.
- `testConnection($ip, $path_private_key)`: Handle deploy rsa key state through `testConnection`.
- `dropKeySsh($params)`: Handle deploy rsa key state through `dropKeySsh`.
- `testkey($param)`: Handle deploy rsa key state through `testkey`.
- `workerDeploy()`: Handle deploy rsa key state through `workerDeploy`.
- `deploy($param)`: Handle deploy rsa key state through `deploy`.
- `parseUserKey($elem, $type_key)`: Handle deploy rsa key state through `parseUserKey`.
- `testParseUserKey()`: Handle deploy rsa key state through `testParseUserKey`.
- `queue($param)`: Handle deploy rsa key state through `queue`.

Detail

Detail

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Detail.php`
- `format($bytes, $decimals)`: Handle detail state through `format`.

Digest

Digest

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Digest.php`
- `integrate($param)`: Handle digest state through `integrate`.
- `insertDigest($param)`: Handle digest state through `insertDigest`.
- `selectIdfromDigest($param)`: Handle digest state through `selectIdfromDigest`.

Disk

Disk

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Disk.php`
- `getData($param)`: Retrieve disk state through `getData`.
- `gg($param)`: Handle disk state through `gg`.

Dns

Dns

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Dns.php`
- `check($param)`: Handle dns state through `check`.

Docker

Docker

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Docker.php`
- `install()`: Handle docker state through `install`.
- `uninstall()`: Handle docker state through `uninstall`.
- `getTag($param)`: Retrieve docker state through `getTag`.
- `getImage($param)`: Retrieve docker state through `getImage`.
- `index($param)`: Render docker state through `index`.
- `createInstance($param)`: Create docker state through `createInstance`.
- `linkTagAndImage($param)`: Handle docker state through `linkTagAndImage`.
- `add($param)`: Create docker state through `add`.
- `installMariadb($param)`: Handle docker state through `installMariadb`.
- `before($param)`: Prepare docker state through `before`.
- `password($length)`: Handle docker state through `password`.
- `getCredentials($param)`: Retrieve docker state through `getCredentials`.

Dot3

Dot3

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Dot3.php`
- `before($param)`: Prepare dot3 state through `before`.
- `generateInformation($param)`: Handle dot3 state through `generateInformation`.
- `generateGroupMasterSlave($information)`: Handle dot3 state through `generateGroupMasterSlave`.
- `generateGroupProxySQL($information)`: Handle dot3 state through `generateGroupProxySQL`.
- `generateGroupVip($information)`: Handle dot3 state through `generateGroupVip`.
- `generateGroupGalera($information)`: Handle dot3 state through `generateGroupGalera`.
- `getIdMysqlServerFromGalera($cluster_address)`: Retrieve dot3 state through `getIdMysqlServerFromGalera`.
- `generateGroupMaxScale($information)`: Handle dot3 state through `generateGroupMaxScale`.
- `test2($param)`: Handle dot3 state through `test2`.
- `run($param)`: Handle dot3 state through `run`.
- `saveGraph($id_dot3_information, $file_name, $dot, $group)`: Update dot3 state through `saveGraph`.
- `writeDot()`: Handle dot3 state through `writeDot`.
- `logMissingProxySqlMysqlServers($server, $context)`: Handle dot3 state through `logMissingProxySqlMysqlServers`.
- `array_merge_group($array)`: Handle dot3 state through `array_merge_group`.
- `array_values_recursive($ary)`: Handle dot3 state through `array_values_recursive`.
- `getGroup($param)`: Retrieve dot3 state through `getGroup`.
- `buildLink($param)`: Handle dot3 state through `buildLink`.
- `buildLinkVIP($param)`: Handle dot3 state through `buildLinkVIP`.
- `buildGaleraSstHintLink($param)`: Handle dot3 state through `buildGaleraSstHintLink`.
- `getGaleraSegmentFromNode($node)`: Retrieve dot3 state through `getGaleraSegmentFromNode`.

- `scoreSstDonorCandidate($donorNode, $joinerSegment)`: Handle dot3 state through `scoreSstDonorCandidate`.
- `buildSstEdgeLabel($donorNode, $joinerNode)`: Handle dot3 state through `buildSstEdgeLabel`.
- `estimateSstProgressPercent($donorNode, $joinerNode, $elapsedSec)`: Handle dot3 state through `estimateSstProgressPercent`.
- `estimateSstElapsedSeconds($donorNode, $joinerNode)`: Handle dot3 state through `estimateSstElapsedSeconds`.
- `formatSstElapsedLabel($elapsedSec)`: Handle dot3 state through `formatSstElapsedLabel`.
- `getPositiveIntMetric($node, $key)`: Retrieve dot3 state through `getPositiveIntMetric`.
- `guessGaleraAutoIncrement($servers, $group, $joinerId, $clusterName)`: Handle dot3 state through `guessGaleraAutoIncrement`.
- `buildServer($param)`: Handle dot3 state through `buildServer`.
- `mergeVipDnsDataInInformation($all, $vipServerIds, $date_request)`: Handle dot3 state through `mergeVipDnsDataInInformation`.
- `isVipServer($server)`: Handle dot3 state through `isVipServer`.
- `enrichVipServerForGraph($server, $allServers)`: Handle dot3 state through `enrichVipServerForGraph`.
- `getVipRenderDestinations($server, $allServers)`: Retrieve dot3 state through `getVipRenderDestinations`.
- `resolveVipDestinationId($idDestination, $allServers, $maxDepth)`: Handle dot3 state through `resolveVipDestinationId`.
- `buildVipDestinationLabel($allServers, $idDestination)`: Handle dot3 state through `buildVipDestinationLabel`.
- `getServerPort($server)`: Retrieve dot3 state through `getServerPort`.
- `getOrCreateUnknownProxySqlServer($host, $referenceId)`: Retrieve dot3 state through `getOrCreateUnknownProxySqlServer`.
- `isUnknownProxySqlNode($id_mysql_server)`: Handle dot3 state through `isUnknownProxySqlNode`.
- `linkProxySQLAdmin($param)`: Handle dot3 state through `linkProxySQLAdmin`.
- `linkHostGroup($param)`: Handle dot3 state through `linkHostGroup`.
- `linkMaxScale($param)`: Handle dot3 state through `linkMaxScale`.
- `loadConfigColor()`: Handle dot3 state through `loadConfigColor`.
- `findIdMysqlServer($host, $id_dot3_information, $silent)`: Handle dot3 state through `findIdMysqlServer`.
- `getInformation($id_dot3_information)`: Retrieve dot3 state through `getInformation`.
- `replaceKey($array, $oldKey, $newKey)`: Handle dot3 state through `replaceKey`.
- `legend()`: Handle dot3 state through `legend`.

- `download($param)`: Handle dot3 state through `download`.
- `escapeTooltip($string)`: Handle dot3 state through `escapeTooltip`.
- `purgeAll($param)`: Handle dot3 state through `purgeAll`.
- `show($param)`: Handle dot3 state through `show`.
- `buildGaleraCluster($param)`: Handle dot3 state through `buildGaleraCluster`.
- `extractProviderOption($wsrep_provider_options, $variable)`: Handle dot3 state through `extractProviderOption`.
- `setThemeToServer($theme, $id_mysql_server)`: Handle dot3 state through `setThemeToServer`.
- `reOrderVariable($variables, $filter)`: Handle dot3 state through `reOrderVariable`.
- `getHostGroup($hostgroups)`: Retrieve dot3 state through `getHostGroup`.
- `buildLinkBetweenProxySQL($param)`: Handle dot3 state through `buildLinkBetweenProxySQL`.
- `resolveMaxScaleConnection($maxscale, $maxscale_ip_port)`: Handle dot3 state through `resolveMaxScaleConnection`.
- `splitAddressPort($value)`: Handle dot3 state through `splitAddressPort`.
- `getTunnel($param)`: Retrieve dot3 state through `getTunnel`.

Enum

Enum

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Enum.php`
- `index($param)`: Render enum state through `index`.

Environment

Environment

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Environment.php`
- `index()`: Render environment state through `index`.
- `update()`: Update environment state through `update`.
- `add($param)`: Create environment state through `add`.
- `delete($param)`: Delete environment state through `delete`.

ErrorWeb

ErrorWeb

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/ErrorWeb.php`
- `error404()`: Handle error web state through `error404`.
- `message($param)`: Handle error web state through `message`.

Event

Event

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Event.php`
- `gg($param)`: Handle event state through `gg`.

Explain

Explain

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Explain.php`

Export

Export

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Export.php`
- `generateDump($param)`: Handle export state through `generateDump`.
- `index()`: Render export state through `index`.
- `export_conf()`: Handle export state through `export_conf`.
- `import_conf($param)`: Handle export state through `import_conf`.
- `import($param)`: Handle export state through `import`.
- `test_import($param)`: Handle export state through `test_import`.
- `_export($options)`: Handle export state through `_export`.
- `test_export($param)`: Handle export state through `test_export`.
- `getExportOption()`: Retrieve export state through `getExportOption`.
- `getUniqueKey($table_name)`: Retrieve export state through `getUniqueKey`.
- `encrypt()`: Handle export state through `encrypt`.
- `option()`: Handle export state through `option`.
- `test_dechiffrement($param)`: Handle export state through `test_dechiffrement`.
- `is_gzipped($in)`: Handle export state through `is_gzipped`.

ForeignKey

ForeignKey

- Type: class
- Namespace: `App\Library`
- Source: `App/Library/ForeignKey.php`
- `__construct($db, $database)`: Handle foreign key state through `__construct`.
- `getPath($table_a, $table_b)`: Retrieve foreign key state through `getPath`.

Format

Format

- Type: class
- Namespace: `App\Library`
- Source: `App/Library/Format.php`
- `bytes($bytes, $decimals)`: Handle format state through `bytes`.

Gagman

Gagman

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Gagman.php`
- `index($param)`: Render gagman state through `index`.

Galera

Galera

- Type: trait
- Namespace: `App\Library`
- Source: `App/Library/Galera.php`
- `getGaleraCluster($param)`: Retrieve galera state through `getGaleraCluster`.
- `deduplicateGaleraClustersByNodeId($clusters)`: Handle galera state through `deduplicateGaleraClustersByNodeId`.
- `getAllMemberFromGalera($incomming, $galera_nodes, $group)`: Retrieve galera state through `getAllMemberFromGalera`.
- `getInfoServer($param)`: Retrieve galera state through `getInfoServer`.
- `mappingMaster()`: Handle galera state through `mappingMaster`.
- `createArbitrator()`: Create galera state through `createArbitrator`.
- `getNewId()`: Retrieve galera state through `getNewId`.

GaleraCluster

GaleraCluster

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/GaleraCluster.php`
- `index($param)`: Render galera cluster state through `index`.
- `getInfoGalera($param)`: Retrieve galera cluster state through `getInfoGalera`.
- `setNodeAsPrimary($param)`: Handle galera cluster state through `setNodeAsPrimary`.

Graph

Graph

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Graph.php`
- `index()`: Render graph state through `index`.
- `cache()`: Handle graph state through `cache`.
- `innodb()`: Handle graph state through `innodb`.
- `galera()`: Handle graph state through `galera`.
- `myisam()`: Handle graph state through `myisam`.
- `engine()`: Handle graph state through `engine`.
- `main()`: Render graph state through `main`.
- `gg($param)`: Handle graph state through `gg`.
- `getElems($string)`: Retrieve graph state through `getElems`.
- `generateGraph($data)`: Handle graph state through `generateGraph`.
- `agregate($param)`: Handle graph state through `agregate`.

GraphicCharter

GraphicCharter

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/GraphicCharter.php`
- `show()`: Handle graphic charter state through `show`.

Group

Group

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Group.php`
- `index()`: Render group state through `index`.

Haproxy

Haproxy

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Haproxy.php`
- `refreshConfiguration($param)`: Handle haproxy state through `refreshConfiguration`.
- `parseConfiguration($config)`: Handle haproxy state through `parseConfiguration`.

Home

Home

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Home.php`
- `before($param)`: Prepare home state through `before`.
- `index()`: Render home state through `index`.
- `list_server($param)`: Retrieve home state through `list_server`.

Index

Index

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Index.php`
- `buildCash($param)`: Handle index state through `buildCash`.
- `IsRedundantIndexes($id_mysql_server, $schema_name, $table_name, $index_name)`: Handle index state through `IsRedundantIndexes`.
- `IsUnusedIndexes($id_mysql_server, $schema_name, $table_name, $index_name)`: Handle index state through `IsUnusedIndexes`.
- `dashboard($param)`: Handle index state through `dashboard`.

Install

Install

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Install.php`
- `out($msg, $type)`: Handle install state through `out`.
- `onError()`: Handle install state through `onError`.
- `cmd($cmd, $msg)`: Handle install state through `cmd`.
- `displayResult($msg, $fine)`: Handle install state through `displayResult`.
- `anonymous($function, $msg)`: Handle install state through `anonymous`.
- `index($param)`: Render install state through `index`.
- `prompt($test)`: Handle install state through `prompt`.
- `testMysqlServer()`: Handle install state through `testMysqlServer`.
- `cadre($text, $elem)`: Handle install state through `cadre`.
- `importData($server)`: Handle install state through `importData`.
- `updateConfig($server)`: Update install state through `updateConfig`.
- `updateCache()`: Update install state through `updateCache`.
- `createAdmin($param)`: Create install state through `createAdmin`.
- `createOrganisation($param)`: Create install state through `createOrganisation`.
- `rand_char($length)`: Handle install state through `rand_char`.
- `generate_key()`: Handle install state through `generate_key`.
- `installLanguage($db)`: Handle install state through `installLanguage`.
- `parseConfig($configFile)`: Handle install state through `parseConfig`.
- `testIpPort($hostname, $port)`: Handle install state through `testIpPort`.
- `testMySQL($hostname, $port, $user, $password)`: Handle install state through `testMySQL`.
- `testVectorDB()`: Handle install state through `testVectorDB`.
- `testSpider()`: Handle install state through `testSpider`.
- `testDatabase($database)`: Handle install state through `testDatabase`.
- `createDatabase($database)`: Create install state through `createDatabase`.
- `configMySQL($config)`: Handle install state through `configMySQL`.

- `webroot($param)`: Handle install state through `webroot`.
- `updateVersion()`: Update install state through `updateVersion`.
- `update($param)`: Update install state through `update`.

Integrate

Integrate

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Integrate.php`
- `before($param)`: Prepare integrate state through `before`.
- `show($param)`: Handle integrate state through `show`.
- `getIdTsFile($ts_file)`: Retrieve integrate state through `getIdTsFile`.
- `get_variable()`: Retrieve integrate state through `get_variable`.
- `isFloat($value)`: Handle integrate state through `isFloat`.
- `getTypeOfData($value)`: Retrieve integrate state through `getTypeOfData`.
- `insert_variable($variables_to_insert)`: Handle integrate state through `insert_variable`.
- `insert_value($values)`: Handle integrate state through `insert_value`.
- `insert_slave_value($values, $val)`: Handle integrate state through `insert_slave_value`.
- `linkServerVariable($history, $memory_file)`: Handle integrate state through `linkServerVariable`.
- `convert($id, $revert)`: Handle integrate state through `convert`.
- `getIdMemoryFile($memory_file)`: Retrieve integrate state through `getIdMemoryFile`.
- `integrateAll($param)`: Handle integrate state through `integrateAll`.
- `isJson($string)`: Handle integrate state through `isJson`.
- `evaluate($param)`: Handle integrate state through `evaluate`.
- `purgeAll($param)`: Handle integrate state through `purgeAll`.
- `normalizeSharedMemoryPayload($payload)`: Handle integrate state through `normalizeSharedMemoryPayload`.

Job

Job

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Job.php`
- `index()`: Render job state through `index`.
- `callback($param)`: Handle job state through `callback`.
- `add($param)`: Create job state through `add`.
- `gg($param)`: Handle job state through `gg`.
- `restart($param)`: Handle job state through `restart`.

Layout

Layout

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Layout.php`
- `header($title)`: Handle layout state through `header`.
- `footer()`: Handle layout state through `footer`.
- `headerPma($param)`: Handle layout state through `headerPma`.
- `footerPma()`: Handle layout state through `footerPma`.
- `headerPmacontrol($param)`: Handle layout state through `headerPmacontrol`.
- `footerPmacontrol()`: Handle layout state through `footerPmacontrol`.
- `ariane($param)`: Handle layout state through `ariane`.
- `getMethod()`: Retrieve layout state through `getMethod`.
- `replaceIndex($method)`: Handle layout state through `replaceIndex`.
- `title($params)`: Handle layout state through `title`.

Ldap

Ldap

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Ldap.php`
- `recursiveArraySearchByKey($haystack, $needle, $ret)`: Handle ldap state through `recursiveArraySearchByKey`.
- `index($param)`: Render ldap state through `index`.
- `testLdap($url, $port)`: Handle ldap state through `testLdap`.
- `postToGet($post, $exclude)`: Handle ldap state through `postToGet`.
- `testLdapCredential($url, $port, $bind_dn, $bind_passwd)`: Handle ldap state through `testLdapCredential`.
- `putUl($error)`: Handle ldap state through `putUl`.
- `before($param)`: Prepare ldap state through `before`.
- `log($level, $type, $msg)`: Handle ldap state through `log`.
- `update_group($id_group)`: Update ldap state through `update_group`.
- `change()`: Handle ldap state through `change`.
- `show()`: Handle ldap state through `show`.
- `obtenirRangLePlusHaut()`: Handle ldap state through `obtenirRangLePlusHaut`.
- `OraganiseNiveau()`: Handle ldap state through `OraganiseNiveau`.
- `UpdateConfigFile($var)`: Handle ldap state through `UpdateConfigFile`.
- `updateFromInstall($param)`: Update ldap state through `updateFromInstall`.
- `parseConfig($configFile)`: Handle ldap state through `parseConfig`.
- `requestLdap($command)`: Handle ldap state through `requestLdap`.

Listener

Listener

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Listener.php`
- `load($param)`: Handle listener state through `load`.
- `init($param)`: Handle listener state through `init`.
- `before($param)`: Prepare listener state through `before`.
- `checkAll($param)`: Handle listener state through `checkAll`.
- `check($param)`: Handle listener state through `check`.
- `getUpdateTodo($param)`: Retrieve listener state through `getUpdateTodo`.
- `dispatch($arr)`: Handle listener state through `dispatch`.
- `updateListener($param)`: Update listener state through `updateListener`.
- `updateDatabase($param)`: Update listener state through `updateDatabase`.
- `extract($data)`: Handle listener state through `extract`.
- `updateElem($table_name, $param)`: Update listener state through `updateElem`.
- `test1($param)`: Handle listener state through `test1`.
- `test2($param)`: Handle listener state through `test2`.
- `test4($param)`: Handle listener state through `test4`.
- `afterUpdateVariable($param)`: Handle listener state through `afterUpdateVariable`.
- `resetAll($param)`: Handle listener state through `resetAll`.
- `index()`: Render listener state through `index`.
- `status($param)`: Handle listener state through `status`.
- `splitAndFormat($text)`: Handle listener state through `splitAndFormat`.
- `test5($param)`: Handle listener state through `test5`.
- `purgeAll($param)`: Handle listener state through `purgeAll`.

Llm

Llm

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Llm.php`
- `analyze($param)`: Handle llm state through `analyze`.
- `extractInput($param)`: Handle llm state through `extractInput`.
- `checkLLMAvailability()`: Handle llm state through `checkLLMAvailability`.
- `callLLM($input)`: Handle llm state through `callLLM`.
- `buildSystemPrompt()`: Handle llm state through `buildSystemPrompt`.
- `parseLLMResponse($raw)`: Handle llm state through `parseLLMResponse`.
- `handleNonOkStatus($parsed)`: Handle llm state through `handleNonOkStatus`.

Load

Load

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Load.php`
- `exec($param)`: Handle load state through `exec`.
- `waitPosition($db, $file, $position)`: Handle load state through `waitPosition`.
- `install($db_order)`: Handle load state through `install`.
- `log($sql)`: Handle load state through `log`.
- `getLogAndPos($filename)`: Retrieve load state through `getLogAndPos`.
- `cmd($cmd)`: Handle load state through `cmd`.

Load2

Load2

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Load2.php`
- `exec($param)`: Handle load2 state through `exec`.
- `waitPosition($db, $file, $position)`: Handle load2 state through `waitPosition`.
- `install($db_order)`: Handle load2 state through `install`.
- `log($sql)`: Handle load2 state through `log`.
- `getLogAndPos($filename)`: Retrieve load2 state through `getLogAndPos`.
- `cmd($cmd)`: Handle load2 state through `cmd`.

Log

Log

- Type: class
- Namespace: `App\Library`
- Source: `App/Library/Log.php`
- `get()`: Retrieve log state through `get`.
- `from()`: Handle log state through `from`.

MasterSlave

MasterSlave

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/MasterSlave.php`
- `index()`: Render master slave state through `index`.
- `getTestServer($param)`: Retrieve master slave state through `getTestServer`.
- `AssociateServerByLevel($param)`: Handle master slave state through `AssociateServerByLevel`.
- `compareVersions($a, $b)`: Handle master slave state through `compareVersions`.
- `compareDigit($a, $b)`: Handle master slave state through `compareDigit`.
- `obtenirPlusGrandChiffre($s)`: Handle master slave state through `obtenirPlusGrandChiffre`.
- `generatePair($param)`: Handle master slave state through `generatePair`.
- `configMasterSlave($param)`: Handle master slave state through `configMasterSlave`.
- `randomPassword()`: Handle master slave state through `randomPassword`.
- `setUpDemo($param)`: Handle master slave state through `setUpDemo`.

MaxScale

MaxScale

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/MaxScale.php`
- `index()`: Render max scale state through `index`.
- `curl($param)`: Handle max scale state through `curl`.

Menu

Menu

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Menu.php`
- `show($params)`: Handle menu state through `show`.
- `getSelectedLevelOneMenu($id_menu)`: Retrieve menu state through `getSelectedLevelOneMenu`.

Monitoring

Monitoring

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Monitoring.php`
- `arrays_are_similar($a, $b)`: Handle monitoring state through `arrays_are_similar`.
- `compare($tab_from, $tab_to)`: Handle monitoring state through `compare`.
- `query($param)`: Handle monitoring state through `query`.
- `explain()`: Handle monitoring state through `explain`.
- `getServer()`: Retrieve monitoring state through `getServer`.

Mysql

Mysql

- Type: class
- Namespace: `App\Library`
- Source: `App/Library/Mysql.php`
- `exportAllUser($db_link)`: Handle mysql state through `exportAllUser`.
- `exportUserByUser($db_link)`: Handle mysql state through `exportUserByUser`.
- `onAddMysqlServer($id_mysql_server)`: Handle mysql state through `onAddMysqlServer`.
- `generateMySQLConfig()`: Handle mysql state through `generateMySQLConfig`.
- `addMaxDate($param)`: Create mysql state through `addMaxDate`.
- `getMaster($id_mysql_server, $connection_name)`: Retrieve mysql state through `getMaster`.
- `getDbLink($id_mysql_server, $name)`: Retrieve mysql state through `getDbLink`.
- `addMysqlServer($data)`: Create mysql state through `addMysqlServer`.
- `getId($value, $table_name, $field, $list)`: Retrieve mysql state through `getId`.
- `isPmaControl($ip, $port)`: Handle mysql state through `isPmaControl`.
- `getHostname($name, $data)`: Retrieve mysql state through `getHostname`.
- `selectOrInsert($value, $table_name, $field, $list)`: Handle mysql state through `selectOrInsert`.
- `getServerInfo($id_mysql_server)`: Retrieve mysql state through `getServerInfo`.
- `execMulti($queries, $db_link)`: Handle mysql state through `execMulti`.
- `getListObject($db_link, $database, $type_object)`: Retrieve mysql state through `getListObject`.
- `getCurrentDb($db)`: Retrieve mysql state through `getCurrentDb`.
- `getStructure($db_link, $database, $data, $object)`: Retrieve mysql state through `getStructure`.
- `getRoutineShowCreateQueries($param)`: Retrieve mysql state through `getRoutineShowCreateQueries`.

- `getIdFromDns($dns_port)`: Retrieve mysql state through `getIdFromDns`.
- `testMySQL($param)`: Handle mysql state through `testMySQL`.
- `getRealForeignKey($param)`: Retrieve mysql state through `getRealForeignKey`.
- `getEmptyDatabase($param)`: Retrieve mysql state through `getEmptyDatabase`.
- `getInfoServer($param)`: Retrieve mysql state through `getInfoServer`.
- `createSelectAccount($param)`: Create mysql state through `createSelectAccount`.
- `getIdMySqlServer($param)`: Retrieve mysql state through `getIdMySqlServer`.
- `getRoles($param)`: Retrieve mysql state through `getRoles`.
- `getCreateRoles($param)`: Retrieve mysql state through `getCreateRoles`.
- `getSlave($param)`: Retrieve mysql state through `getSlave`.
- `generateProxySQLConfig()`: Handle mysql state through `generateProxySQLConfig`.
- `execute($id_mysql_server, $file_name)`: Handle mysql state through `execute`.
- `test($hostname, $port, $user, $password)`: Handle mysql state through `test`.
- `test2($hostname, $port, $user, $password)`: Handle mysql state through `test2`.
- `getIdMySQLFromGalera($wsrep_incoming_addresses)`: Retrieve mysql state through `getIdMySQLFromGalera`.
- `getIdMysqlServerFromIpPort($ip, $port)`: Retrieve mysql state through `getIdMysqlServerFromIpPort`.
- `getNameMysqlServerFromIpPort($ip, $port)`: Retrieve mysql state through `getNameMysqlServerFromIpPort`.

MysqlDatabase

MysqlDatabase

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/MysqlDatabase.php`
- `addNoDatabaseCautionFlash()`: Create mysql database state through `addNoDatabaseCautionFlash`.
- `menu($param)`: Handle mysql database state through `menu`.
- `getDatabaseByServer($param)`: Retrieve mysql database state through `getDatabaseByServer`.
- `mpd($param)`: Handle mysql database state through `mpd`.
- `foreignKey($param)`: Handle mysql database state through `foreignKey`.
- `table($param)`: Handle mysql database state through `table`.

MysqlTable

MysqlTable

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/MysqlTable.php`
- `menu($param)`: Handle mysql table state through `menu`.
- `getTableByDatabase($param)`: Retrieve mysql table state through `getTableByDatabase`.

MysqlUser

MysqlUser

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/MysqlUser.php`
- `index($param)`: Render mysql user state through `index`.
- `backup()`: Handle mysql user state through `backup`.
- `cmpHost($param)`: Handle mysql user state through `cmpHost`.
- `security($param)`: Handle mysql user state through `security`.
- `getUserNeverConnected($param)`: Retrieve mysql user state through `getUserNeverConnected`.
- `role($param)`: Handle mysql user state through `role`.
- `export($param)`: Handle mysql user state through `export`.

Mysqlsys

Mysqlsys

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Mysqlsys.php`
- `index()`: Render mysqlsys state through `index`.
- `install()`: Handle mysqlsys state through `install`.
- `addFormat($tab)`: Create mysqlsys state through `addFormat`.
- `reset($param)`: Handle mysqlsys state through `reset`.
- `drop($param)`: Handle mysqlsys state through `drop`.
- `updateConfig($param)`: Update mysqlsys state through `updateConfig`.
- `export($param)`: Handle mysqlsys state through `export`.

Myxplain

Myxplain

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Myxplain.php`
- `index($param)`: Render myxplain state through `index`.
- `import()`: Handle myxplain state through `import`.

Ollama

Ollama

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Ollama.php`
- `index($param)`: Render ollama state through `index`.

Partition

Partition

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Partition.php`
- `generate($param)`: Handle partition state through `generate`.

Percona

Percona

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Percona.php`
- `execQuery($param)`: Handle percona state through `execQuery`.
- `getServeAvailable($param)`: Retrieve percona state through `getServeAvailable`.
- `ptOsc($param)`: Handle percona state through `ptOsc`.
- `updateOsc($param)`: Update percona state through `updateOsc`.
- `delOldOscTable($param)`: Handle percona state through `delOldOscTable`.
- `displayOsc($param)`: Handle percona state through `displayOsc`.
- `delAllOldOscTable($param)`: Handle percona state through `delAllOldOscTable`.

PhpLiveRegex

PhpLiveRegex

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/PhpLiveRegex.php`
- `index()`: Render php live regex state through `index`.
- `evaluate($param)`: Handle php live regex state through `evaluate`.
- `pregView($regex, $options, $replace, $data)`: Handle php live regex state through `pregView`.

Pid

Pid

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Pid.php`
- `index()`: Render pid state through `index`.
- `deleteOldPid($param)`: Delete pid state through `deleteOldPid`.

Plugin

Plugin

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Plugin.php`
- `index($param)`: Render plugin state through `index`.
- `jsontodatabase($jsonInText)`: Handle plugin state through `jsontodatabase`.
- `install($param)`: Handle plugin state through `install`.
- `copyfile($file, $source, $target, $nest)`: Handle plugin state through `copyfile`.
- `logpluginfile($handle, $pluginid, $source, $target)`: Handle plugin state through `logpluginfile`.
- `sqlexecute($filename)`: Handle plugin state through `sqlexecute`.
- `remove($param)`: Delete plugin state through `remove`.

Pmacontrol

Pmacontrol

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Pmacontrol.php`
- `before($param)`: Prepare pmacontrol state through `before`.
- `setPageMeta($title, $ariane, $meta)`: Handle pmacontrol state through `setPageMeta`.
- `index()`: Render pmacontrol state through `index`.
- `product()`: Handle pmacontrol state through `product`.
- `monitoring()`: Handle pmacontrol state through `monitoring`.
- `performance()`: Handle pmacontrol state through `performance`.
- `backups()`: Handle pmacontrol state through `backups`.
- `galera()`: Handle pmacontrol state through `galera`.
- `proxysql()`: Handle pmacontrol state through `proxysql`.
- `schema()`: Handle pmacontrol state through `schema`.
- `security()`: Handle pmacontrol state through `security`.
- `automation()`: Handle pmacontrol state through `automation`.
- `solutions()`: Handle pmacontrol state through `solutions`.
- `integrations()`: Handle pmacontrol state through `integrations`.
- `pricing()`: Handle pmacontrol state through `pricing`.
- `docs()`: Handle pmacontrol state through `docs`.
- `resources()`: Handle pmacontrol state through `resources`.
- `blog()`: Handle pmacontrol state through `blog`.
- `blog_article()`: Handle pmacontrol state through `blog_article`.
- `case_studies()`: Handle pmacontrol state through `case_studies`.
- `whitepapers()`: Handle pmacontrol state through `whitepapers`.
- `webinars()`: Handle pmacontrol state through `webinars`.
- `company()`: Handle pmacontrol state through `company`.
- `roadmap()`: Handle pmacontrol state through `roadmap`.
- `security_page()`: Handle pmacontrol state through `security_page`.
- `contact()`: Handle pmacontrol state through `contact`.
- `privacy()`: Handle pmacontrol state through `privacy`.

- `terms()`: Handle pmacontrol state through `terms`.
- `cookies()`: Handle pmacontrol state through `cookies`.
- `ai()`: Handle pmacontrol state through `ai`.

Pmm

Pmm

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Pmm.php`
- `export()`: Handle pmm state through `export`.

PostMortem

PostMortem

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/PostMortem.php`
- `format($bytes, $decimals)`: Handle post mortem state through `format`.

ProxySQL

ProxySQL

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/ProxySQL.php`
- `before($param)`: Prepare proxy s q l state through `before`.
- `main($param)`: Render proxy s q l state through `main`.
- `add()`: Create proxy s q l state through `add`.
- `testProxySQLAdmin($param)`: Handle proxy s q l state through `testProxySQLAdmin`.
- `index($param)`: Render proxy s q l state through `index`.
- `getServers($hostname, $port, $login, $password)`: Retrieve proxy s q l state through `getServers`.
- `associate($param)`: Handle proxy s q l state through `associate`.
- `statistic($param)`: Handle proxy s q l state through `statistic`.
- `getErrorConnect($param)`: Retrieve proxy s q l state through `getErrorConnect`.
- `import($param)`: Handle proxy s q l state through `import`.
- `auto($param)`: Handle proxy s q l state through `auto`.
- `getConfigMenuDefinition()`: Retrieve proxy s q l state through `getConfigMenuDefinition`.
- `extractTableNameFromSqlTemplate($sql_template)`: Handle proxy s q l state through `extractTableNameFromSqlTemplate`.
- `getSqliteCreateTableStatement($db, $table_name)`: Retrieve proxy s q l state through `getSqliteCreateTableStatement`.
- `normalizeDefaultValue($default_value)`: Handle proxy s q l state through `normalizeDefaultValue`.
- `parseEnumValues($raw_values)`: Handle proxy s q l state through `parseEnumValues`.
- `getEnumValuesByColumn($create_table_sql, $columns)`: Retrieve proxy s q l state through `getEnumValuesByColumn`.
- `getAutoincrementByColumn($create_table_sql, $columns)`: Retrieve proxy s q l state through `getAutoincrementByColumn`.
- `config($param)`: Handle proxy s q l state through `config`.
- `update($param)`: Update proxy s q l state through `update`.
- `menu($param)`: Handle proxy s q l state through `menu`.

- `ifProxySqlExist($param)`: Handle proxy s q l state through `ifProxySqlExist`.
- `insertProxySqlAdmin($param)`: Handle proxy s q l state through `insertProxySqlAdmin`.
- `updateField($param)`: Update proxy s q l state through `updateField`.
- `deleteLine($param)`: Delete proxy s q l state through `deleteLine`.
- `monitor($param)`: Handle proxy s q l state through `monitor`.
- `cluster($param)`: Handle proxy s q l state through `cluster`.
- `log($param)`: Handle proxy s q l state through `log`.
- `getIdMysqlServer($param)`: Retrieve proxy s q l state through `getIdMysqlServer`.
- `addLine($param)`: Create proxy s q l state through `addLine`.

Query

Query

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Query.php`
- `getFielsWithoutDefault($id_mysql_server, $databases)`: Retrieve query state through `getFielsWithoutDefault`.
- `getDefaultValueByType($type, $typeExtra)`: Retrieve query state through `getDefaultValueByType`.
- `setDefault($param)`: Handle query state through `setDefault`.
- `dropDefault($param)`: Handle query state through `dropDefault`.
- `runSetDefault($param)`: Handle query state through `runSetDefault`.
- `runQuery($param)`: Handle query state through `runQuery`.
- `createWorkTable($param)`: Create query state through `createWorkTable`.
- `deleteWorkTable($param)`: Delete query state through `deleteWorkTable`.
- `sigHandler($signo)`: Handle query state through `sigHandler`.
- `getMaxRun($param)`: Retrieve query state through `getMaxRun`.
- `byDigest($param)`: Handle query state through `byDigest`.
- `extractTablesFromSQL($sql)`: Handle query state through `extractTablesFromSQL`.
- `extractTablesAndAliases($sql)`: Handle query state through `extractTablesAndAliases`.
- `extractTablesWithOffsets($sql)`: Handle query state through `extractTablesWithOffsets`.
- `extract($param)`: Handle query state through `extract`.
- `normalize_sql_for_digest($sql, $ansi_quotes)`: Handle query state through `normalize_sql_for_digest`.
- `normalize_sql_strict($sql, $ansi_quotes)`: Handle query state through `normalize_sql_strict`.
- `statement_digest_text($sql, $max_digest_length, $perf_max_digest_length, $ansi_quotes)`: Handle query state through `statement_digest_text`.
- `statement_digest($sql, $max_digest_length, $ansi_quotes)`: Handle query state through `statement_digest`.
- `statement_digest_text_strict($sql, $max_digest_length, $show_length, $ansi_quotes)`: Handle query state through `statement_digest_text_strict`.

- `statement_digest_strict($sql, $max_digest_length, $ansi_quotes)`: Handle query state through `statement_digest_strict`.
- `testDigest($param)`: Handle query state through `testDigest`.
- `digest2($query)`: Handle query state through `digest2`.
- `digestText($query)`: Handle query state through `digestText`.
- `normalize($query)`: Handle query state through `normalize`.
- `isKeyword($word)`: Handle query state through `isKeyword`.
- `replaceAlias($string)`: Handle query state through `replaceAlias`.
- `collectDigest($param)`: Handle query state through `collectDigest`.
- `collectAll($param)`: Handle query state through `collectAll`.
- `degradeMariaDB($string)`: Handle query state through `degradeMariaDB`.

QueryCache

QueryCache

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/QueryCache.php`
- `index($param)`: Render query cache state through `index`.

QueryGraphExtractor

QueryGraphExtractor

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/QueryGraphExtractor.php`
- `extractSelectBlock($sql)`: Handle query graph extractor state through `extractSelectBlock`.
- `splitSelectFields($selectBlock)`: Handle query graph extractor state through `splitSelectFields`.
- `normalizeSelectField($field, $order)`: Handle query graph extractor state through `normalizeSelectField`.
- `extractSelectFields($sql)`: Handle query graph extractor state through `extractSelectFields`.
- `extractTables($sql)`: Handle query graph extractor state through `extractTables`.
- `extractJoinBlocks($sql)`: Handle query graph extractor state through `extractJoinBlocks`.
- `extractJoinConditions($condBlock)`: Handle query graph extractor state through `extractJoinConditions`.
- `extractJoins($sql)`: Handle query graph extractor state through `extractJoins`.
- `extractWhereBlock($sql)`: Handle query graph extractor state through `extractWhereBlock`.
- `extractWhereConditions($sql)`: Handle query graph extractor state through `extractWhereConditions`.
- `extract($sql)`: Handle query graph extractor state through `extract`.
- `extractSubqueryBlocks($sql)`: Handle query graph extractor state through `extractSubqueryBlocks`.
- `extractSubquery($sql, $alias)`: Handle query graph extractor state through `extractSubquery`.
- `extractSubqueries($sql)`: Handle query graph extractor state through `extractSubqueries`.
- `extractGroupBy($sql)`: Handle query graph extractor state through `extractGroupBy`.

Recover

Recover

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Recover.php`
- `rewrite($param)`: Handle recover state through `rewrite`.
- `getTableId($table_schema, $table_name)`: Retrieve recover state through `getTableId`.
- `getTableName($file)`: Retrieve recover state through `getTableName`.
- `importData($param)`: Handle recover state through `importData`.
- `removeComments($content)`: Delete recover state through `removeComments`.

Release

Release

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Release.php`
- `make($params)`: Handle release state through `make`.
- `bdd()`: Handle release state through `bdd`.
- `getLastVersion()`: Retrieve release state through `getLastVersion`.
- `publishVersion()`: Handle release state through `publishVersion`.
- `getOldsql()`: Retrieve release state through `getOldsql`.

Replication

Replication

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Replication.php`
- `index()`: Render replication state through `index`.
- `status()`: Handle replication state through `status`.
- `event()`: Handle replication state through `event`.

Scan

Scan

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Scan.php`
- `parse($input)`: Handle scan state through `parse`.
- `autoDiscovering()`: Handle scan state through `autoDiscovering`.
- `extract($data)`: Handle scan state through `extract`.
- `xmlToArray($xml)`: Handle scan state through `xmlToArray`.
- `index()`: Render scan state through `index`.
- `__sleep()`: Handle scan state through `__sleep`.
- `getData($refresh)`: Retrieve scan state through `getData`.
- `generateNmap($range)`: Handle scan state through `generateNmap`.
- `getIpMonitored()`: Retrieve scan state through `getIpMonitored`.
- `refresh()`: Handle scan state through `refresh`.
- `generateRange($ips)`: Handle scan state through `generateRange`.
- `scanner()`: Handle scan state through `scanner`.
- `scanner2($ranges)`: Handle scan state through `scanner2`.
- `pingAll($param)`: Handle scan state through `pingAll`.
- `testPing($host, $timeout)`: Handle scan state through `testPing`.
- `generateListIps($ranges)`: Handle scan state through `generateListIps`.
- `collectHosts()`: Handle scan state through `collectHosts`.
- `scanAllPort($combi)`: Handle scan state through `scanAllPort`.
- `generatePortForScan($ips)`: Handle scan state through `generatePortForScan`.
- `scanPort($ip, $port, $maxExecutionTime)`: Handle scan state through `scanPort`.
- `scanAll($param)`: Handle scan state through `scanAll`.
- `save()`: Update scan state through `save`.

Schema

Schema

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Schema.php`
- `export($param)`: Handle schema state through `export`.
- `importScript($param)`: Handle schema state through `importScript`.
- `parseImportScriptOptions($args)`: Handle schema state through `parseImportScriptOptions`.
- `buildImportTables($basePath)`: Handle schema state through `buildImportTables`.
- `buildImportViews($basePath)`: Handle schema state through `buildImportViews`.
- `buildImportProcedures($basePath)`: Handle schema state through `buildImportProcedures`.
- `buildImportFunctions($basePath)`: Handle schema state through `buildImportFunctions`.
- `buildImportTriggers($basePath)`: Handle schema state through `buildImportTriggers`.
- `buildImportEvents($basePath)`: Handle schema state through `buildImportEvents`.
- `concatImportChunks($chunks)`: Handle schema state through `concatImportChunks`.
- `loadSqlFiles($path)`: Handle schema state through `loadSqlFiles`.
- `buildSqlSection($label, $files)`: Handle schema state through `buildSqlSection`.
- `buildRoutineSection($label, $files)`: Handle schema state through `buildRoutineSection`.
- `readSqlFile($path)`: Handle schema state through `readSqlFile`.
- `formatRoutineFile($path)`: Handle schema state through `formatRoutineFile`.
- `exportRoutines($id_mysql_server, $database, $basePath, $routineType)`: Handle schema state through `exportRoutines`.
- `exportTriggers($id_mysql_server, $database, $basePath)`: Handle schema state through `exportTriggers`.
- `exportEvents($id_mysql_server, $database, $basePath)`: Handle schema state through `exportEvents`.

- `executeRoutineShowCreate($id_mysql_server, $sqlShow, $routineType)`: Handle schema state through `executeRoutineShowCreate`.
- `extractRoutineName($row, $definitionKey, $routineType)`: Handle schema state through `extractRoutineName`.
- `ensureDirectory($path)`: Handle schema state through `ensureDirectory`.
- `ensureSchemaDirectoryStructure($basePath)`: Handle schema state through `ensureSchemaDirectoryStructure`.
- `initializeGitRepository($path)`: Handle schema state through `initializeGitRepository`.
- `ensureGitRepository($path)`: Handle schema state through `ensureGitRepository`.
- `commitSchemaSnapshot($idMysqlServer, $database, $path, $serverMeta)`: Handle schema state through `commitSchemaSnapshot`.
- `getGitStatus($path, $exclude)`: Retrieve schema state through `getGitStatus`.
- `commitSubDirectorySnapshot($basePath, $subDirectory, $database, $serverMeta, $label)`: Handle schema state through `commitSubDirectorySnapshot`.
- `ensureGitSafeDirectory($path)`: Handle schema state through `ensureGitSafeDirectory`.
- `removeAutoIncrement($createStatement)`: Delete schema state through `removeAutoIncrement`.
- `getNextSnapshotNumber($path)`: Retrieve schema state through `getNextSnapshotNumber`.
- `cleanupObsoleteSchemaFiles($path, $folder, $currentObjects)`: Handle schema state through `cleanupObsoleteSchemaFiles`.
- `extractCreateViewStatement($createRow)`: Handle schema state through `extractCreateViewStatement`.
- `ensureCreateOrReplaceView($statement)`: Handle schema state through `ensureCreateOrReplaceView`.
- `normalizeLineEndings($sql)`: Handle schema state through `normalizeLineEndings`.
- `runDos2Unix($path)`: Handle schema state through `runDos2Unix`.
- `parseGitStatus($status)`: Handle schema state through `parseGitStatus`.
- `notifySchemaChange($database, $snapshotNumber, $changes, $serverMeta)`: Handle schema state through `notifySchemaChange`.
- `formatChangeList($items)`: Handle schema state through `formatChangeList`.
- `formatServerLabel($meta)`: Handle schema state through `formatServerLabel`.
- `isUnknownDatabaseError($exception)`: Handle schema state through `isUnknownDatabaseError`.

- `exportAll($param)`: Handle schema state through `exportAll`.
- `exportSchemasForServer($id_mysql_server)`: Handle schema state through `exportSchemasForServer`.
- `getEligibleServerIds($param)`: Retrieve schema state through `getEligibleServerIds`.
- `watchLoop($param)`: Handle schema state through `watchLoop`.
- `loadDdlState()`: Handle schema state through `loadDdlState`.
- `saveDdlState($state)`: Update schema state through `saveDdlState`.
- `getDdlStateFile()`: Retrieve schema state through `getDdlStateFile`.
- `toInt($value)`: Handle schema state through `toInt`.
- `computeDelta($previous, $current)`: Handle schema state through `computeDelta`.
- `compareModels($param)`: Handle schema state through `compareModels`.
- `diffModelServers($leftId, $rightId, $options)`: Handle schema state through `diffModelServers`.
- `getModelServerPath($serverId)`: Retrieve schema state through `getModelServerPath`.
- `listModelDatabases($serverPath)`: Retrieve schema state through `listModelDatabases`.
- `listModelObjects($databasePath)`: Retrieve schema state through `listModelObjects`.
- `listModelSqlFiles($databasePath)`: Retrieve schema state through `listModelSqlFiles`.
- `diffModelDatabase($leftDatabasePath, $rightDatabasePath, $ignoreColumnOrder)`: Handle schema state through `diffModelDatabase`.
- `formatModelComparison($comparison)`: Handle schema state through `formatModelComparison`.
- `compareModelsUi($param)`: Handle schema state through `compareModelsUi`.
- `buildComparisonDetails($differences, $ignoreColumnOrder)`: Handle schema state through `buildComparisonDetails`.
- `renderDiffTable($leftFile, $rightFile, $ignoreColumnOrder)`: Handle schema state through `renderDiffTable`.
- `getDiffClass($type)`: Retrieve schema state through `getDiffClass`.
- `getDiffPrefix($type)`: Retrieve schema state through `getDiffPrefix`.
- `readFileContent($path)`: Handle schema state through `readFileContent`.
- `areModelFilesIdentical($fileA, $fileB, $ignoreColumnOrder)`: Handle schema state through `areModelFilesIdentical`.
- `normalizeCreateTableStatement($sql)`: Handle schema state through `normalizeCreateTableStatement`.

- `splitSqlDefinitionList($body)`: Handle schema state through `splitSqlDefinitionList`.
- `isColumnDefinitionLine($definition)`: Handle schema state through `isColumnDefinitionLine`.
- `isIndexDefinitionLine($definition)`: Handle schema state through `isIndexDefinitionLine`.
- `extractColumnName($definition)`: Handle schema state through `extractColumnName`.
- `extractIndexName($definition)`: Handle schema state through `extractIndexName`.
- `isEscapedByBackslash($subject, $position)`: Handle schema state through `isEscapedByBackslash`.
- `getDiffTableCss()`: Retrieve schema state through `getDiffTableCss`.
- `getDiffer()`: Retrieve schema state through `getDiffer`.
- `watch($param)`: Handle schema state through `watch`.
- `migration($param)`: Handle schema state through `migration`.
- `migrationAll($param)`: Handle schema state through `migrationAll`.
- `migrateSchemaRepo($param)`: Handle schema state through `migrateSchemaRepo`.
- `migrateSchemaReposAll($param)`: Handle schema state through `migrateSchemaReposAll`.
- `listSchemaRepoSkips($param)`: Retrieve schema state through `listSchemaRepoSkips`.
- `runMigration($basePath, $serverFilter)`: Handle schema state through `runMigration`.
- `formatMigrationSummary($summary)`: Handle schema state through `formatMigrationSummary`.
- `moveFile($source, $destination)`: Handle schema state through `moveFile`.
- `moveDirectory($source, $destination)`: Handle schema state through `moveDirectory`.
- `copyDirectory($source, $destination)`: Handle schema state through `copyDirectory`.
- `removeDirectory($path)`: Delete schema state through `removeDirectory`.

Server

Server

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Server.php`
- `before($param)`: Prepare server state through `before`.
- `hardware()`: Handle server state through `hardware`.
- `main($param)`: Render server state through `main`.
- `database()`: Handle server state through `database`.
- `statistics()`: Handle server state through `statistics`.
- `logs()`: Handle server state through `logs`.
- `buildQuery($fields)`: Handle server state through `buildQuery`.
- `memory()`: Handle server state through `memory`.
- `index()`: Render server state through `index`.
- `id($param)`: Handle server state through `id`.
- `settings()`: Handle server state through `settings`.
- `getClients()`: Retrieve server state through `getClients`.
- `getEnvironments()`: Retrieve server state through `getEnvironments`.
- `add()`: Create server state through `add`.
- `cache()`: Handle server state through `cache`.
- `passwd($param)`: Handle server state through `passwd`.
- `show($param)`: Handle server state through `show`.
- `updateHostname()`: Update server state through `updateHostname`.
- `box()`: Handle server state through `box`.
- `password($param)`: Handle server state through `password`.
- `acknowledge($param)`: Handle server state through `acknowledge`.
- `remove($param)`: Delete server state through `remove`.
- `acknowledgedBy($param)`: Handle server state through `acknowledgedBy`.
- `toggleGeneralLog($param)`: Toggle server state through `toggleGeneralLog`.
- `getDaemonRunning($param)`: Retrieve server state through `getDaemonRunning`.
- `testGetDaemonRunning($param)`: Handle server state through `testGetDaemonRunning`.

- `retract($param)`: Handle server state through `retract`.
- `getFlag($param)`: Retrieve server state through `getFlag`.
- `ssl($param)`: Handle server state through `ssl`.

Site

Site

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Site.php`
- `before($param)`: Prepare site state through `before`.
- `buildCommon()`: Handle site state through `buildCommon`.
- `setActive($id)`: Handle site state through `setActive`.
- `index()`: Render site state through `index`.
- `saas()`: Handle site state through `saas`.
- `onpremise()`: Handle site state through `onpremise`.
- `agents()`: Handle site state through `agents`.
- `max()`: Handle site state through `max`.
- `offers()`: Handle site state through `offers`.
- `integrations()`: Handle site state through `integrations`.
- `documentation()`: Handle site state through `documentation`.
- `faq()`: Handle site state through `faq`.
- `process()`: Handle site state through `process`.
- `demo()`: Handle site state through `demo`.
- `resources()`: Handle site state through `resources`.
- `blog()`: Handle site state through `blog`.
- `support()`: Handle site state through `support`.
- `contact()`: Handle site state through `contact`.
- `roadmap()`: Handle site state through `roadmap`.
- `success()`: Handle site state through `success`.
- `lab()`: Handle site state through `lab`.
- `incidents()`: Handle site state through `incidents`.

Slave

Slave

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Slave.php`
- `index()`: Render slave state through `index`.
- `generateGraph($slaves)`: Handle slave state through `generateGraph`.
- `show($param)`: Handle slave state through `show`.
- `box()`: Handle slave state through `box`.
- `generateGraphSlave($slaves)`: Handle slave state through `generateGraphSlave`.
- `startSlave($param)`: Handle slave state through `startSlave`.
- `stopSlave($param)`: Handle slave state through `stopSlave`.
- `setSlave($param)`: Handle slave state through `setSlave`.
- `getInfoServer($id_mysql_server)`: Retrieve slave state through `getInfoServer`.
- `runInBackground($command, $log, $priority)`: Handle slave state through `runInBackground`.
- `isProcessRunning($PID)`: Handle slave state through `isProcessRunning`.
- `getMasterInfo($param)`: Retrieve slave state through `getMasterInfo`.
- `switchOver($param)`: Handle slave state through `switchOver`.
- `activateGtid($param)`: Handle slave state through `activateGtid`.
- `deactivateGtid($param)`: Handle slave state through `deactivateGtid`.
- `skipCounter($param)`: Handle slave state through `skipCounter`.
- `extractBinlogInfo($line)`: Handle slave state through `extractBinlogInfo`.
- `groupBinlogInfoByPosition($extractedData)`: Handle slave state through `groupBinlogInfoByPosition`.
- `processBinlogFile($param)`: Handle slave state through `processBinlogFile`.
- `generateCmd($param)`: Handle slave state through `generateCmd`.
- `execute($sql, $db, $DRY_RUN)`: Handle slave state through `execute`.
- `generateSecurePassword($length)`: Handle slave state through `generateSecurePassword`.
- `waitForSlavePosition($param)`: Handle slave state through `waitForSlavePosition`.

Spider

Spider

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Spider.php`
- `index()`: Render spider state through `index`.
- `Server($param)`: Handle spider state through `Server`.
- `testIfSpiderExist($param)`: Handle spider state through `testIfSpiderExist`.
- `getServerLink($id_mysql_server)`: Retrieve spider state through `getServerLink`.
- `extractSpiderInfoFromCreateTable($createTable)`: Handle spider state through `extractSpiderInfoFromCreateTable`.
- `create()`: Create spider state through `create`.
- `addLinkDb($param)`: Create spider state through `addLinkDb`.

Ssh

Ssh

- Type: class
- Namespace: `App\Library`
- Source: `App/Library/Ssh.php`
- `setMockInstance($instance)`: Handle ssh state through `setMockInstance`.
- `formatPrivateKey($key)`: Handle ssh state through `formatPrivateKey`.
- `connect($ip, $port, $user, $password)`: Handle ssh state through `connect`.
- `close()`: Handle ssh state through `close`.
- `isValid($pubkeyssh)`: Handle ssh state through `isValid`.
- `generate($type, $bit)`: Handle ssh state through `generate`.
- `put($server, $port, $login, $private_key, $src, $dst)`: Handle ssh state through `put`.
- `spaceAvailable($param)`: Handle ssh state through `spaceAvailable`.
- `ssh($id_server, $type)`: Handle ssh state through `ssh`.
- `getSsh($id, $type)`: Retrieve ssh state through `getSsh`.

StatementAnalysis

StatementAnalysis

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/StatementAnalysis.php`
- `index($param)`: Render statement analysis state through `index`.

StorageArea

StorageArea

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/StorageArea.php`
- `index($param)`: Render storage area state through `index`.
- `add($param)`: Create storage area state through `add`.
- `listStorage()`: Retrieve storage area state through `listStorage`.
- `getStorageSpace($param)`: Retrieve storage area state through `getStorageSpace`.
- `delete($param)`: Delete storage area state through `delete`.
- `menu($param)`: Handle storage area state through `menu`.
- `update($param)`: Update storage area state through `update`.

Table

Table

- Type: class
- Namespace: `App\Library`
- Source: `App/Library/Table.php`
- `getTableDefinition($param)`: Retrieve table state through `getTableDefinition`.
- `findFieldPosition($param)`: Handle table state through `findFieldPosition`.
- `getNumberOfField($param)`: Retrieve table state through `getNumberOfField`.
- `getCount($param)`: Retrieve table state through `getCount`.
- `getIndex($param)`: Retrieve table state through `getIndex`.
- `importRealForeignKey($param)`: Handle table state through `importRealForeignKey`.
- `getTableWithFk($param)`: Retrieve table state through `getTableWithFk`.

Tag

Tag

- Type: class
- Namespace: `App\Library`
- Source: `App/Library/Tag.php`
- `insertTag($id_mysql_server, $all_tags)`: Handle tag state through `insertTag`.

Telegram

Telegram

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Telegram.php`
- `index($param)`: Render telegram state through `index`.
- `add()`: Create telegram state through `add`.
- `view($param)`: Handle telegram state through `view`.
- `delete($param)`: Delete telegram state through `delete`.

Translation

Translation

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Translation.php`
- `index()`: Render translation state through `index`.
- `admin_translation()`: Handle translation state through `admin_translation`.
- `delete_tmp_files()`: Delete translation state through `delete_tmp_files`.
- `delete_table_cach()`: Delete translation state through `delete_table_cach`.
- `getNew($param)`: Retrieve translation state through `getNew`.
- `askApiGoogle($param)`: Handle translation state through `askApiGoogle`.
- `settings($param)`: Handle translation state through `settings`.
- `before($param)`: Prepare translation state through `before`.

Tree

Tree

- Type: class
- Namespace: `App\Library`
- Source: `App/Library/Tree.php`
- `__construct($db_link, $table_name, $fields, $options)`: Handle tree state through `__construct`.
- `delete($id)`: Delete tree state through `delete`.
- `extraWhere()`: Handle tree state through `extraWhere`.
- `add($leaf, $id_parent)`: Create tree state through `add`.
- `up($id)`: Handle tree state through `up`.
- `countFather($id)`: Handle tree state through `countFather`.
- `getInterval($id)`: Retrieve tree state through `getInterval`.
- `getfather($id)`: Retrieve tree state through `getfather`.
- `left($id)`: Handle tree state through `left`.
- `removeaclfile()`: Delete tree state through `removeaclfile`.
- `getFirstFather($id)`: Retrieve tree state through `getFirstFather`.

Tunnel

Tunnel

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Tunnel.php`
- `agent($param)`: Handle tunnel state through `agent`.

Upgrade

Upgrade

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Upgrade.php`
- `now($param)`: Handle upgrade state through `now`.
- `setNewVersion($param)`: Handle upgrade state through `setNewVersion`.
- `executePatch($param)`: Handle upgrade state through `executePatch`.
- `getPatchFrom($build)`: Retrieve upgrade state through `getPatchFrom`.
- `needUpgrade($param)`: Handle upgrade state through `needUpgrade`.
- `updateConfig($param)`: Update upgrade state through `updateConfig`.

User

User

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/User.php`
- `before($param)`: Prepare user state through `before`.
- `after($param)`: Handle user state through `after`.
- `index()`: Render user state through `index`.
- `is_logged()`: Handle user state through `is_logged`.
- `block_newsletter()`: Handle user state through `block_newsletter`.
- `city()`: Handle user state through `city`.
- `author()`: Handle user state through `author`.
- `register()`: Handle user state through `register`.
- `lost_password()`: Handle user state through `lost_password`.
- `password_recover($param)`: Handle user state through `password_recover`.
- `block_last_registered()`: Handle user state through `block_last_registered`.
- `block_last_online()`: Handle user state through `block_last_online`.
- `admin_user()`: Handle user state through `admin_user`.
- `confirmation($data)`: Handle user state through `confirmation`.
- `login($login, $password)`: Handle user state through `login`.
- `log($id_user, $success)`: Handle user state through `log`.
- `profil($param)`: Handle user state through `profil`.
- `user_main()`: Handle user state through `user_main`.
- `settings($param)`: Handle user state through `settings`.
- `photo($param)`: Handle user state through `photo`.
- `get_new_mail()`: Retrieve user state through `get_new_mail`.
- `send_confirmation()`: Handle user state through `send_confirmation`.
- `connection()`: Handle user state through `connection`.
- `logout()`: Handle user state through `logout`.
- `updateGroup()`: Update user state through `updateGroup`.
- `login($login, $password)`: Handle user state through `login`.
- `update_idgroup()`: Update user state through `update_idgroup`.

Variable

Variable

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Variable.php`
- `index($param)`: Render variable state through `index`.
- `tsVariable($param)`: Handle variable state through `tsVariable`.

Ventilateur

Ventilateur

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Ventilateur.php`
- `queue()`: Handle ventilateur state through `queue`.
- `worker()`: Handle ventilateur state through `worker`.
- `pull()`: Handle ventilateur state through `pull`.
- `add()`: Create ventilateur state through `add`.

Version

Version

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Version.php`
- `index()`: Render version state through `index`.

Webservice

Webservice

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Webservice.php`
- `pushServer($param)`: Handle webservice state through `pushServer`.
- `importFile($param)`: Handle webservice state through `importFile`.
- `checkCredentials($user, $password)`: Handle webservice state through `checkCredentials`.
- `parseServer($filename)`: Handle webservice state through `parseServer`.
- `parseTest($param)`: Handle webservice state through `parseTest`.
- `saveHistory($id_user_main, $json)`: Update webservice state through `saveHistory`.
- `addAccount($param)`: Create webservice state through `addAccount`.
- `index()`: Render webservice state through `index`.
- `parseConfig($configFile)`: Handle webservice state through `parseConfig`.
- `decrypt($param)`: Handle webservice state through `decrypt`.
- `isJson($string)`: Handle webservice state through `isJson`.

Worker

Worker

- Type: class
- Namespace: `App\Controller`
- Source: `App/Controller/Worker.php`
- `before($param)`: Prepare worker state through `before`.
- `logger($param)`: Handle worker state through `logger`.
- `run($param)`: Handle worker state through `run`.
- `keepConfigFile($param)`: Handle worker state through `keepConfigFile`.
- `adaptNumberWorker($param)`: Handle worker state through `adaptNumberWorker`.
- `summarizeAvailability($data)`: Handle worker state through `summarizeAvailability`.
- `generateWorkerUpdateQueries($availability)`: Handle worker state through `generateWorkerUpdateQueries`.
- `test($param)`: Handle worker state through `test`.
- `index($param)`: Render worker state through `index`.
- `checkAll($param)`: Handle worker state through `checkAll`.
- `check($param)`: Handle worker state through `check`.
- `addWorker($param)`: Create worker state through `addWorker`.
- `removeWorker($param)`: Delete worker state through `removeWorker`.
- `killAll($param)`: Handle worker state through `killAll`.
- `addToQueue($param)`: Create worker state through `addToQueue`.
- `update()`: Update worker state through `update`.
- `dropAllQueue($param)`: Handle worker state through `dropAllQueue`.
- `file($param)`: Handle worker state through `file`.
- `getPidWorking($param)`: Retrieve worker state through `getPidWorking`.
- `checkPid($param)`: Handle worker state through `checkPid`.
- `deleteWorkerPid($param)`: Delete worker state through `deleteWorkerPid`.
- `getRunningId($param)`: Retrieve worker state through `getRunningId`.
- `getListofWorkingServer($param)`: Retrieve worker state through `getListofWorkingServer`.

- `deleteExpiredPid($param)`: Delete worker state through `deleteExpiredPid`.